



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра практической и прикладной информатики (ППИ)

ОТЧЕТ ПО ПРАКТИЧЕСКИМ РАБОТАМ

по дисциплине «Разработка обеспечивающих подсистем»

Тема: Проектирование информационной системы сопровождения заявки
концепта клиента на ремонт квартиры.

Студент группы ИНБО-12-23, Губарев С.А.

(подпись)

Преподаватель Трушин С.М.

(подпись)

Москва 2026 г.

Оглавление

Практическое занятие № 1	3
Практическое занятие № 2	10
Практическое занятие № 3	18
Практическое занятие № 4	28
Практическое занятие № 5	36
Практическое занятие № 6	46
Практическое занятие № 7	61
Практическое занятие № 8	79

Практическое занятие № 1

Цель занятия: проанализировать текущие процессы, выявить проблемы исполнения бизнес-процесса, сформировать требования к информационной системе (FR/NFR).

Постановка задачи:

1. Построить модель текущего состояния бизнес-процесса (AS IS) в нотациях IDEF0 и BPMN 2.0.
2. Заполнить таблицы функциональных (FR) и нефункциональных (NFR) требований.
3. Выполнить сравнительный анализ программных аналогов и построить таблицы и диаграммы сравнения ПО.
4. Использовать инструменты (Draw.io, Bizagi Modeler, Sila Union, Stormbpmn, текстовый и табличный редактор) для визуализации моделей и подготовки таблиц.

Результат работы:

1. Описание процесса AS-IS

В исходной учебной модели покупатель направляет пожелания менеджеру в форме переписки и файлов. Менеджер вручную передаёт материалы дизайнеру. Концепция и спецификация последовательно согласуются с покупателем, проектным отделом и отделом строительства. При замечаниях обновлённые файлы рассылаются повторно. После согласования дизайнер или менеджер вручную передаёт перечень материалов в снабжение.

Проблемы AS-IS: отсутствует единый номер и статус заявки; возможна работа с устаревшей версией концепции; реквизиты повторно вводятся в разные документы; замечания не имеют общего журнала; контроль сроков и полноты закупочной потребности выполняется вручную.

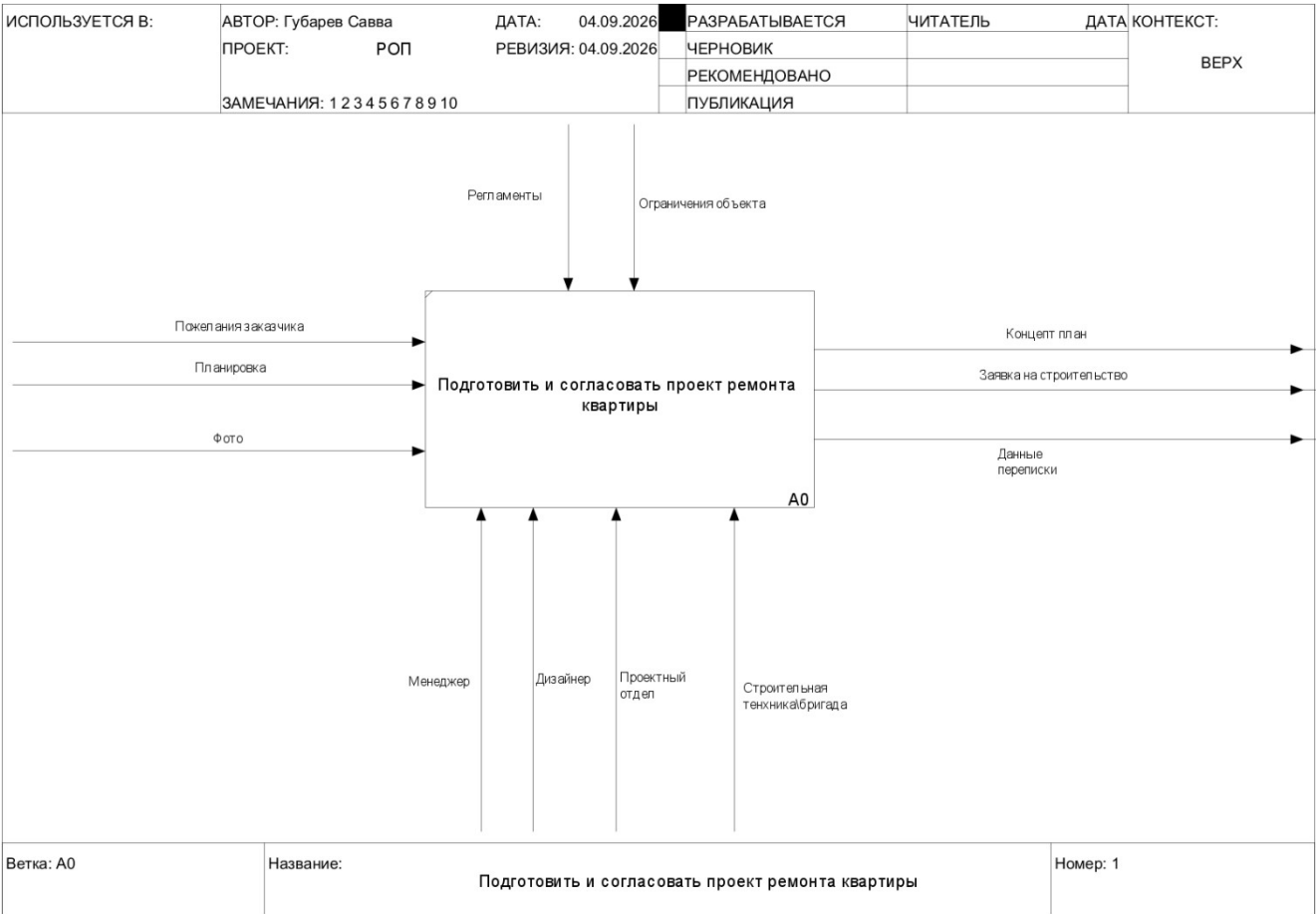


Рисунок 1 - Контекстная модель IDEF0 текущего процесса

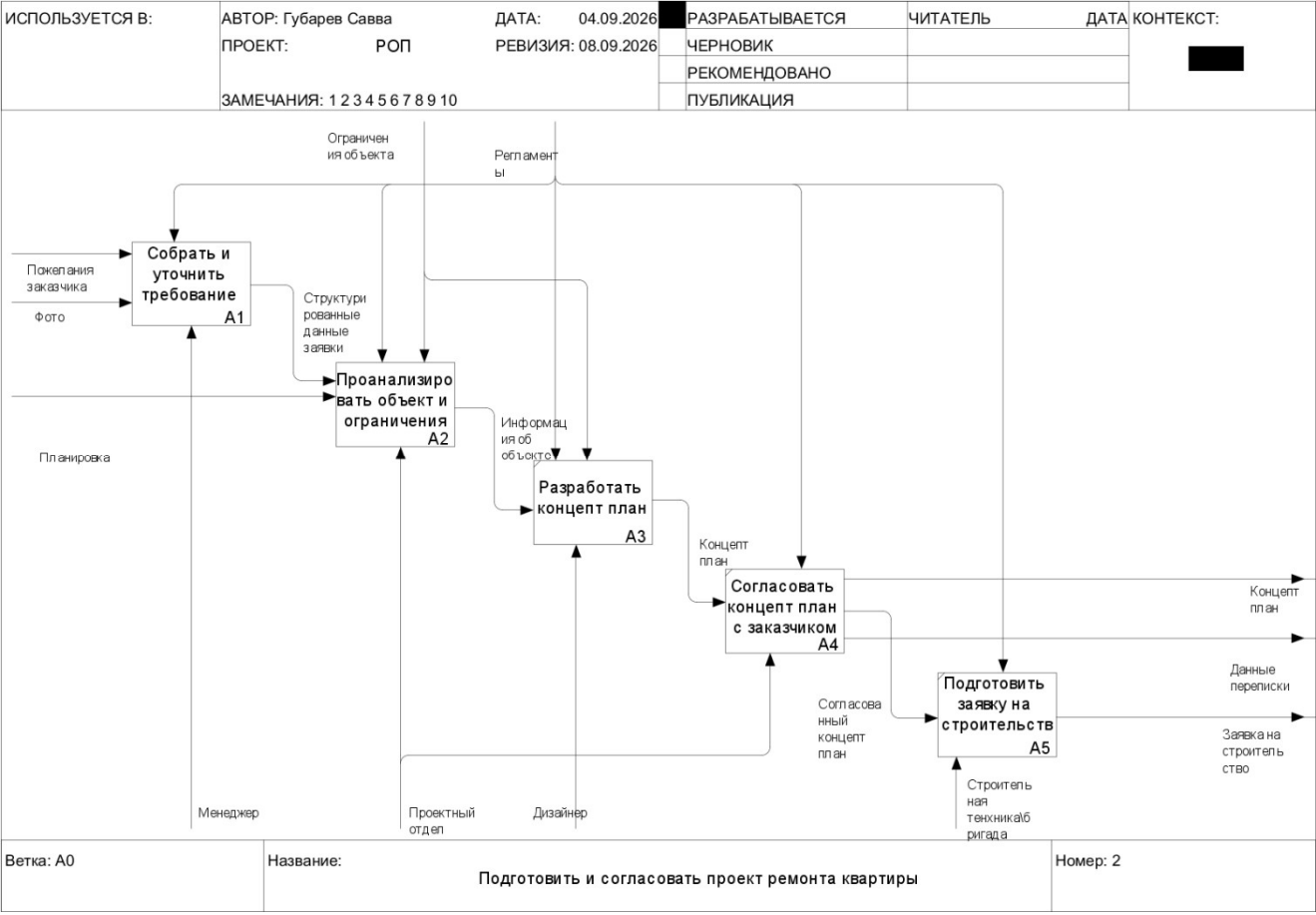


Рисунок 2 - Декомпозиция процесса

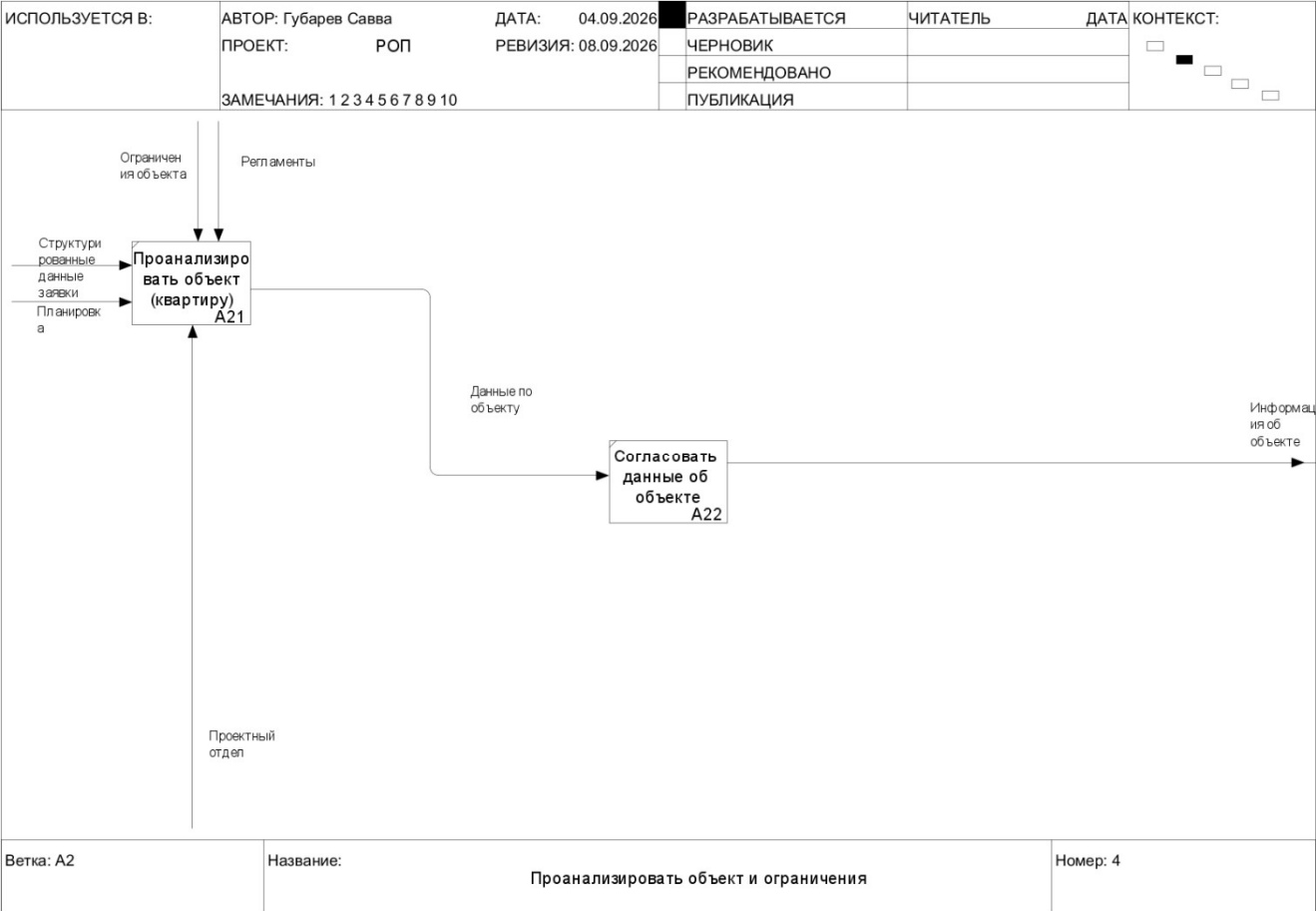


Рисунок 3 - Декомпозиция «Проанализировать объект и ограничения»

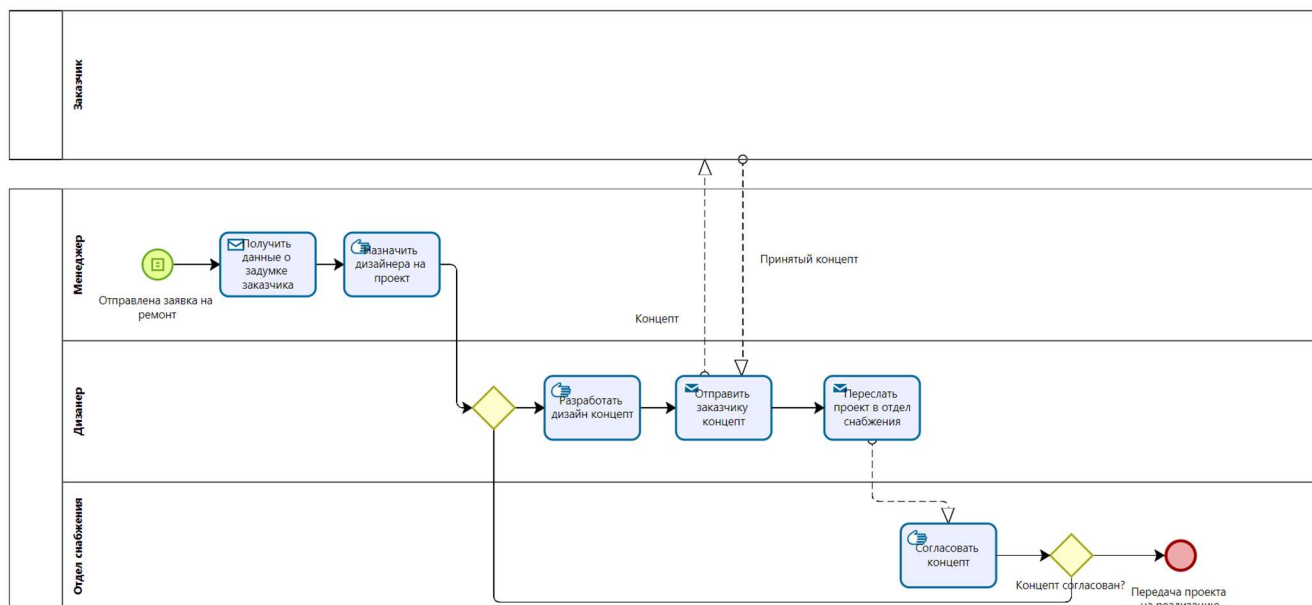


Рисунок 4 - Упрощённая BPMN-модель текущего процесса

2. Функциональные требования

Таблица 1 - Функциональные требования к ИС

Код	Функция	Проверяемое требование
FR-01	Анкета покупателя	Система должна сохранять объект, квартиру, стиль, материалы, планировку, бюджет и комментарии; обязательные поля проверяются.
FR-02	Регистрация	При отправке анкеты система должна создать заявку с уникальным номером, датой и статусом.
FR-03	Назначение	Менеджер должен назначать дизайнера; назначенному сотруднику направляется уведомление.
FR-04	Концепция	Дизайнер должен добавлять описание, планировочные файлы, изображения и спецификацию материалов.
FR-05	Версии	При корректировке концепции система должна создавать новую версию и сохранять предыдущую.
FR-06	Согласование	Система должна собирать решения покупателя, проектного отдела и строительства по одной версии.
FR-07	Доработка	Согласующий должен возвращать концепцию с обязательным замечанием; дизайнер получает уведомление.
FR-08	Закупка	Из утверждённой спецификации система должна формировать закупочную потребность с артикулами, количествами и сроком.
FR-09	Статусы	Система должна проверять допустимые переходы статусов и показывать покупателю только его заявки.
FR-10	Завершение	Уполномоченный сотрудник должен фиксировать завершение и фактически применённые материалы.
FR-11	Аналитика	Система должна рассчитывать популярность товаров по завершённым проектам за период.
FR-12	Документы и поиск	Система должна формировать документы и искать заявки по номеру, квартире, статусу, ответственному и периоду.
FR-13	Администрирование	Администратор должен управлять ролями и справочниками; действия по заявке фиксируются в журнале.

3. Нефункциональные требования

Таблица 2 - Нефункциональные требования к ИС

Код	Категория	Измеримое требование
NFR-01	Производительность	95% операций просмотра карточки и поиска выполняются не более чем за 2 с при 100 одновременных пользователей.
NFR-02	Доступность	Доступность web-системы должна составлять не менее 95% за календарный месяц, кроме согласованных окон обслуживания.
NFR-03	Резервное копирование	Полная резервная копия БД создаётся не реже одного раза в сутки; восстановление тестовой копии - не реже одного раза в квартал.
NFR-04	Восстановление	После подтверждённого сбоя восстановление данных и работоспособности должно занимать не более 4 часов.
NFR-05	Аутентификация	Внутренние пользователи входят по индивидуальной учётной записи; пароль хранится в виде стойкого хеша с солью.
NFR-06	Авторизация	Права доступа проверяются для каждой операции; покупатель не может просматривать заявки других покупателей.
NFR-07	Аудит	Журнал действий должен хранить автора, время, объект, действие и прежнее/новое значение не менее 3 лет.
NFR-08	Совместимость	Клиентская часть должна работать в актуальных двух последних версиях Chrome, Edge и Firefox при разрешении от 1280 px.
NFR-09	Удобство	Покупатель должен создать и отправить анкету не более чем за 7 экранных шагов без обязательного обучения.
NFR-10	Масштабируемость	Система должна поддерживать хранение не менее 100 000 заявок без изменения пользовательского функционала.

4. Сравнительный анализ программных аналогов

Анализ выполнен как учебное сравнение открыто заявленных функций. Planoplan и Remplanner специализированы на подготовке интерьерных решений и спецификаций; Битрикс24 удобен для карточек, задач и маршрутов, но не содержит предметной модели ремонта. Проектируемая ИС объединяет именно маршрут заявки, контроль версий, межфункциональные согласования и передачу потребности в снабжение.

Таблица 3 - Сравнение программных аналогов

ПО	Основное назначение	Концепция и спецификация	Маршрут согласования	Версии	Закупочная потребность	Аналитика ремонта
Planoplan	3D-планировка, материалы, спецификация	Да	Нет	Нет	Нет	Нет
Remplanner	Чертежи, смета, 3D-визуализация	Да	Нет	Нет	Нет	Нет
Битрикс24	CRM, задачи, бизнес-процессы	Нет	Да	Нет	Да	Нет
Проектируемая ИС	Заявка на ремонт и сквозной маршрут	Да	Да	Да	Да	Да

Вывод: целесообразна разработка специализированной ИС, использующей сильные стороны аналогов, но обеспечивающей единый контролируемый маршрут от пожеланий покупателя до снабжения.

Практическое занятие № 2

Цель занятия: спроектировать целевой бизнес-процесс, архитектуру системы и сценарии использования.

Постановка задачи:

1. Построить модель целевого бизнес-процесса (TO BE) в нотациях IDEF0 и BPMN 2.0.
2. Составить Function Tree (дерево функций) для описания функциональной структуры системы.
3. Построить Component Diagram для отображения архитектуры системы.
4. Разработать сценарии использования системы (Use Cases).
5. Использовать инструменты (Draw.io, PlantUML, ArchiMate, текстовый и табличный редактор) для визуализации моделей и оформления таблиц.

Результат работы:

1. Модель целевого процесса ТО-ВЕ

В целевом процессе покупатель заполняет анкету в web-интерфейсе. ИС регистрирует заявку и ведёт единый статус. Менеджер назначает ответственного дизайнера, после чего система фиксирует назначение и направляет уведомление. Дизайнер формирует версию концепции и спецификацию. Система направляет задания на согласование покупателю, проектному отделу и строительству. При отрицательном решении создаётся цикл доработки с обязательным комментарием. После всех положительных решений система формирует закупочную потребность и передаёт её сотруднику снабжения.

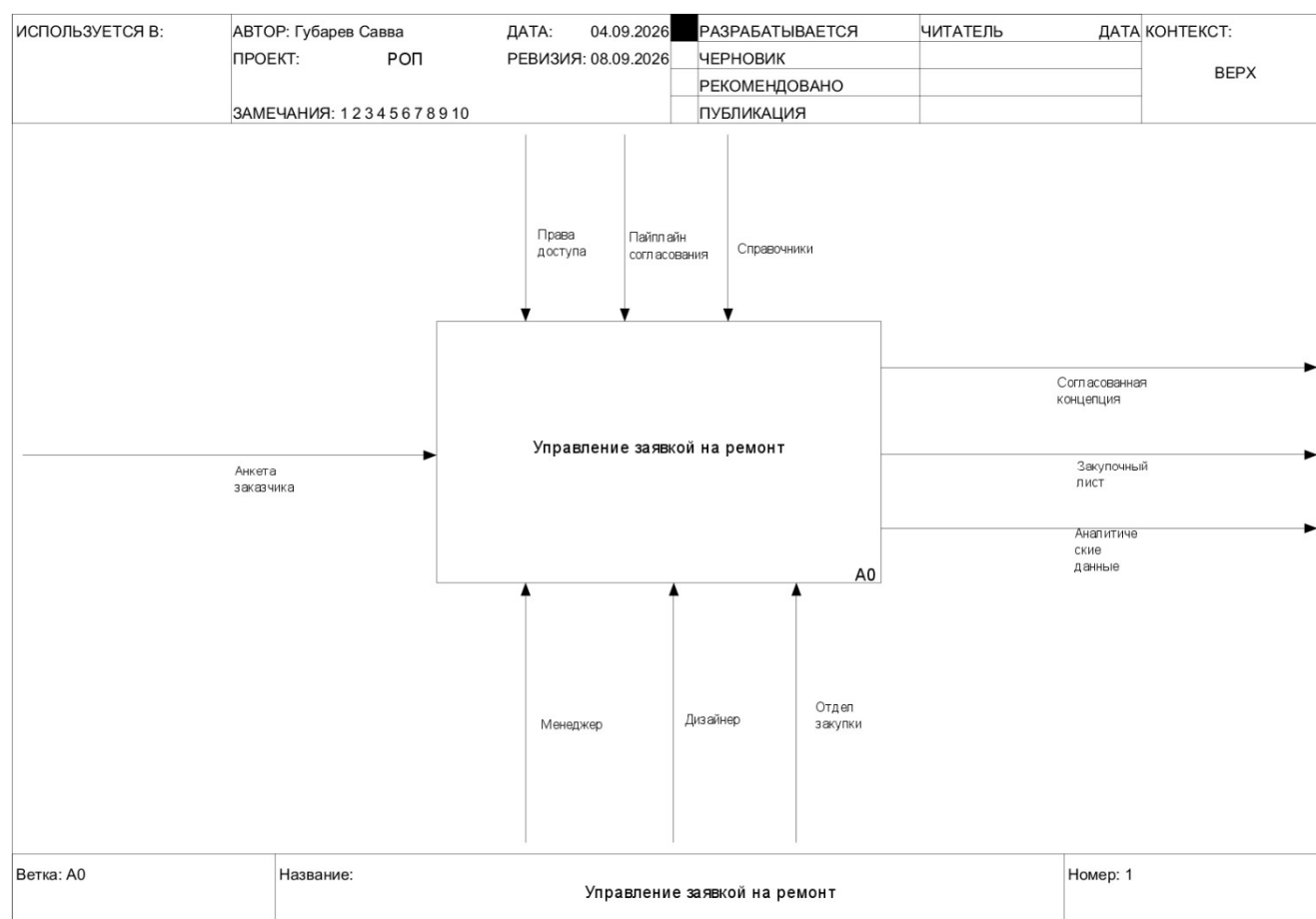


Рисунок 5 - Контекстная модель IDEF0 целевого процесса

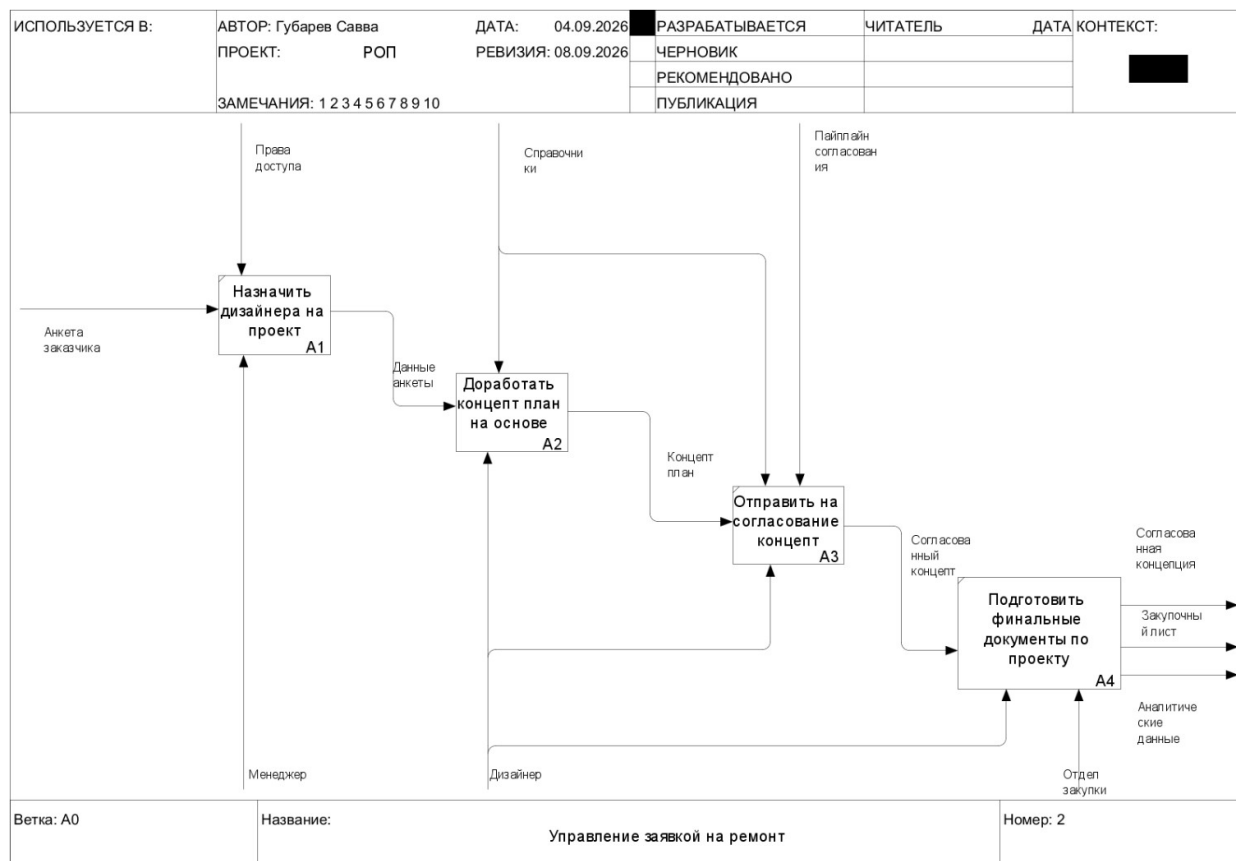


Рисунок 6 - Декомпозиция процесса

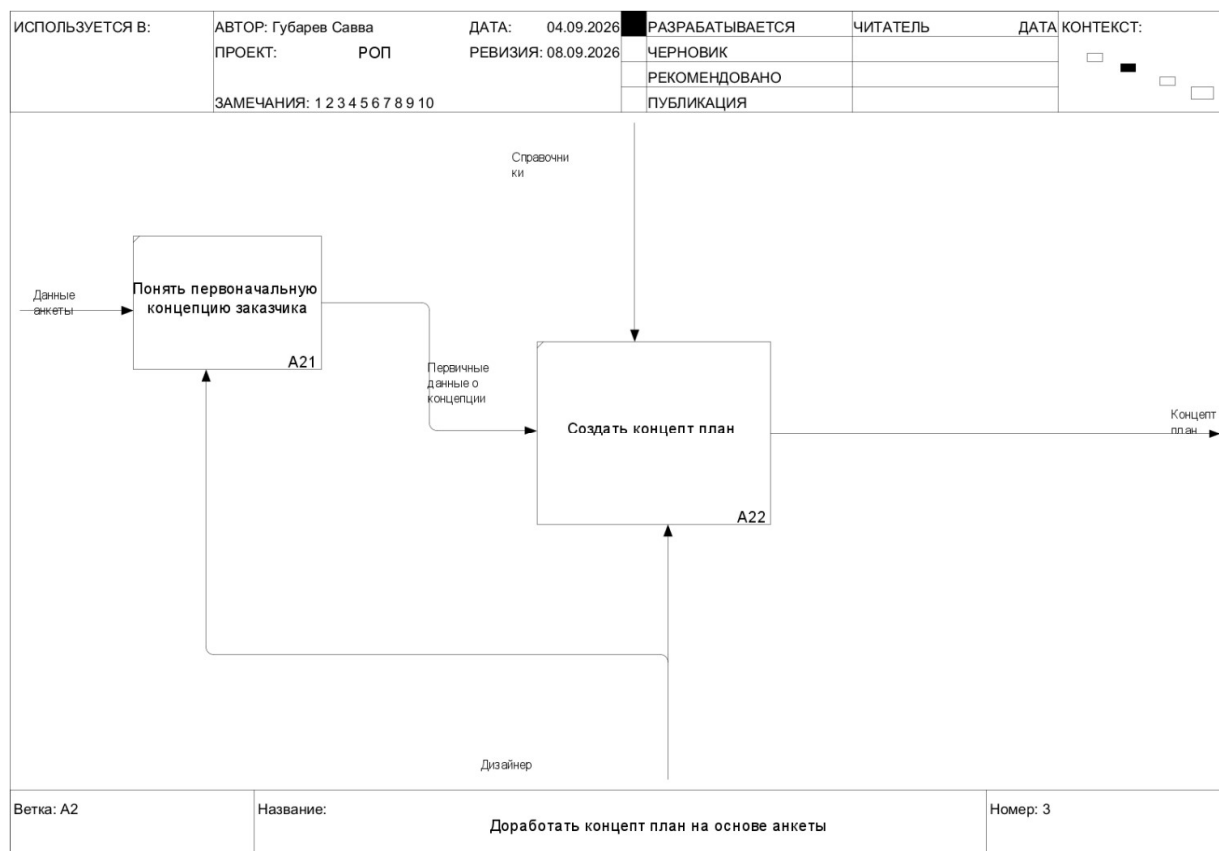


Рисунок 7 - Декомпозиция «Доработать концепт план на основе анкеты»

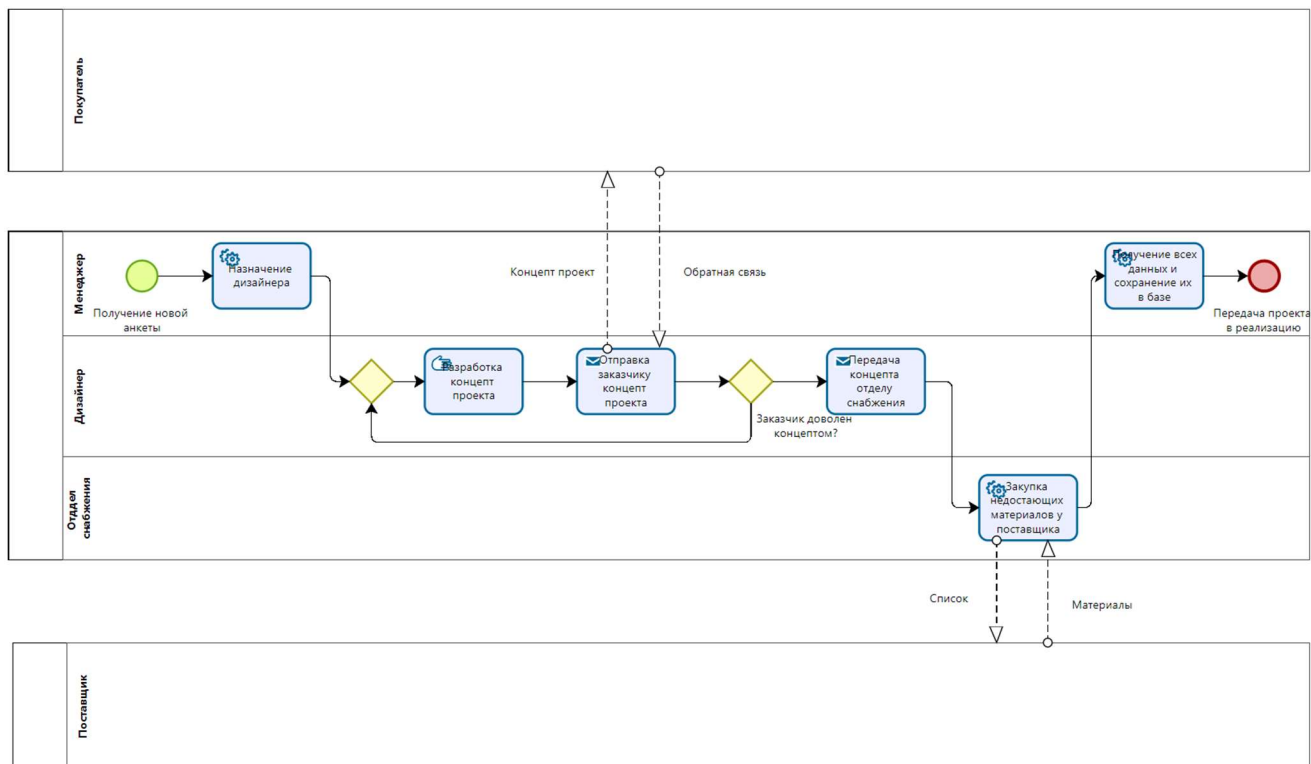


Рисунок 8 - BPMN-модель целевого процесса

2. Дерево функций

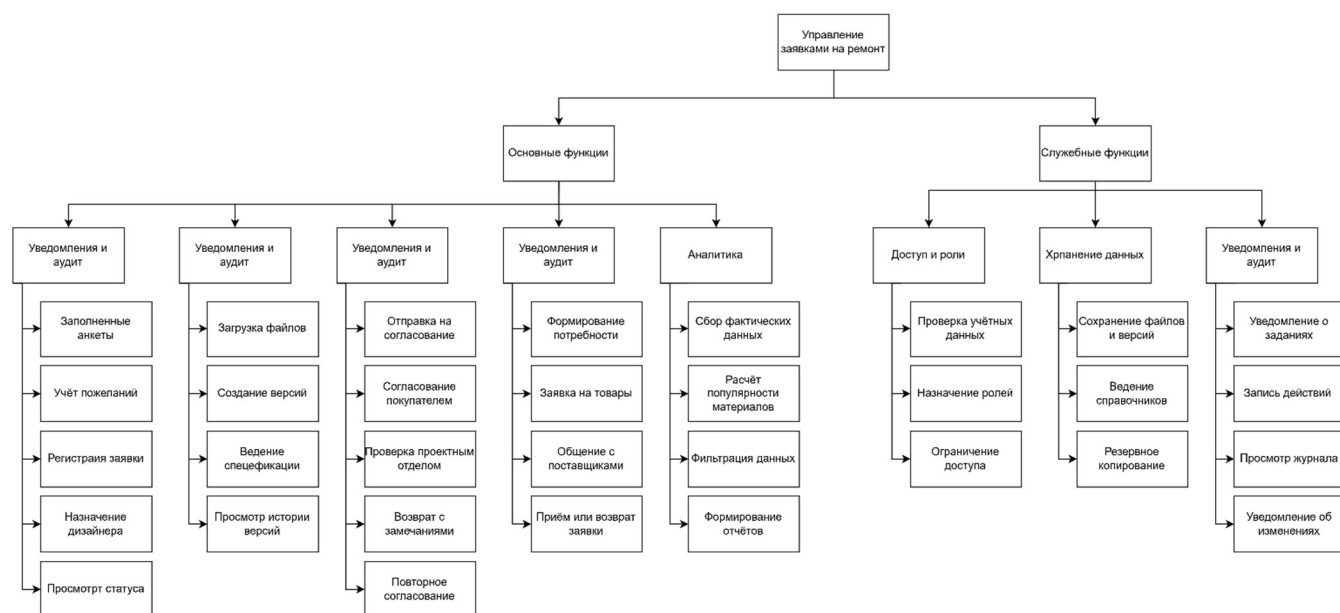


Рисунок 9 - Дерево функций ИС

Таблица 4 - Описание функций системы

Функция	Подфункции	Ответственный	Вход	Выход
Приём заявки	Заполнение анкеты; учёт пожеланий; регистрация заявки; назначение дизайнера; просмотр статуса	Покупатель, менеджер	Анкета, сведения об объекте, пожелания покупателя	Карточка заявки, назначенный дизайнер, статус заявки
Подготовка концепции	Загрузка файлов; создание версий; ведение спецификации; просмотр истории версий	Дизайнер	Заявка, данные объекта, справочники материалов	Версия концепции, файлы проекта, спецификация
Согласование	Отправка на согласование; проверка покупателем, проектным отделом и строительством; возврат с замечаниями; повторное согласование	Покупатель, проектный отдел, отдел строительства	Версия концепции и спецификация	Утверждённая версия либо замечания для доработки
Снабжение	Формирование потребности; подготовка заявки на товары; взаимодействие с поставщиками; приём или возврат заявки	Менеджер, отдел снабжения	Утверждённая спецификация, сведения о поставщиках	Закупочная потребность, принятая в работу, либо причина возврата
Аналитика	Сбор фактических данных; расчёт популярности материалов; фильтрация данных; формирование отчётов	Аналитик	Данные завершённых ремонтов, фактически использованные материалы, параметры выборки	Статистика использования материалов, аналитический отчёт
Доступ и роли	Проверка учётных данных; назначение ролей; ограничение доступа	Администратор	Учётные данные пользователей, роли, правила доступа	Результат проверки доступа, назначенные права
Хранение данных	Сохранение файлов и версий; ведение справочников; резервное копирование	Администратор	Данные заявок, файлы концепций, версии, справочники	Сохранённые данные, актуальные справочники, резервные копии
Уведомления и аудит	Уведомления о заданиях и изменениях; запись действий; просмотр журнала	ИС автоматически; администратор контролирует	События системы, действия пользователей, сведения о получателях	Уведомления, записи журнала действий

3. Компонентная архитектура



Рисунок 10 - Диаграмма компонентов проектируемой ИС

Web-интерфейсы предоставляют покупателю и сотрудникам доступ по HTTPS. Сервис заявок реализует карточку, версии и правила статусов; сервис согласований управляет заданиями и решениями. Данные и аудит хранятся в базе данных, файлы концепции - в файловом хранилище, уведомления рассылаются отдельным сервисом.

4. Сценарии использования

Таблица 5 - Сценарии использования

Сценарий	Акторы	Предусловия	Основные действия	Результат
UC-01. Создать заявку	Покупатель	Покупатель авторизован; объект доступен.	Покупатель заполняет анкету; система проверяет поля, создаёт номер и уведомляет менеджера.	Заявка зарегистрирована.
UC-02. Подготовить концепцию	Дизайнер	Дизайнер назначен на заявку.	Дизайнер добавляет файлы, описание и спецификацию; система сохраняет версию.	Концепция направлена на согласование.
UC-03. Согласовать концепцию	Покупатель, проектный отдел, строительство	Есть актуальная версия концепции.	Каждый участник принимает решение; при замечании указывает комментарий.	Версия согласована либо возвращена на доработку.
UC-04. Сформировать закупочную потребность	Снабжение	Все обязательные решения положительны.	Система формирует позиции из утверждённой спецификации; снабжение подтверждает приём или возвращает с причиной.	Потребность принята в работу снабжением.
UC-05. Получить аналитику	Аналитик	Есть завершённые проекты за период.	Аналитик выбирает период, объект и категорию; система агрегирует фактически применённые материалы.	Построен отчёт о популярности товаров.

5. Вывод

Спроектированный ТО-ВЕ процесс переводит работу с заявкой в единую систему: устраняет разрозненность файлов, фиксирует версии и решения, контролирует маршрут согласования и создаёт полную закупочную потребность только по утверждённой концепции. Модели и сценарии служат основой для дальнейшего проектирования данных и интерфейсов.

Практическое занятие № 3

Цель занятия:

Разработать логическую модель базы данных информационной системы сопровождения заявки на ремонт квартиры, определить сущности, их атрибуты и связи, выделить первичные и внешние ключи, а также разработать кодификаторы и правила формирования идентификационных кодов.

Согласно заданию практического занятия необходимо построить ERD, определить ключевые поля, разработать таблицы кодов и схему кодирования данных.

Постановка задачи:

1. Построить логическую модель базы данных проектируемой информационной системы.
2. Определить основные сущности, их атрибуты и взаимосвязи.
3. Определить первичные и внешние ключи.
4. Разработать кодификаторы для унификации значений статусов, ролей и категорий.
5. Разработать схему кодирования заявок.
6. Построить ER-диаграмму проектируемой базы данных.

Результат работы:

1. Разработка логической модели базы данных

Логическая модель базы данных сформирована на основании функций информационной системы, определённых в предыдущих практических занятиях. Система должна обеспечивать регистрацию заявок, назначение ответственных сотрудников, хранение версий концепции, согласование проекта, ведение спецификаций, формирование закупочной потребности, уведомления и аудит действий пользователей. Эти функции ранее были зафиксированы в функциональных требованиях FR-01-FR-13.

Для хранения данных выделены следующие основные сущности.

Таблица 6 - Сущности и атрибуты базы данных

Сущность	Основные атрибуты	Первичный ключ	Внешние ключи
Пользователь	user_id, ФИО, email, телефон, password_hash, role_code	user_id	role_code
Роль	role_code, наименование, описание	role_code	-
Объект ремонта	object_id, customer_id, адрес, площадь, количество комнат, описание	object_id	customer_id
Заявка	application_id, application_code, object_id, customer_id, manager_id, designer_id, status_code, дата создания, бюджет, комментарий	application_id	object_id, customer_id, manager_id, designer_id, status_code
Статус заявки	status_code, наименование, описание	status_code	-
Концепция	concept_id, application_id, номер текущей версии, дата создания	concept_id	application_id
Версия концепции	version_id, concept_id, version_number, описание, created_by, дата создания, признак актуальности	version_id	concept_id, created_by

Файл проекта	file_id, version_id, имя файла, путь хранения, тип файла, дата загрузки	file_id	version_id
Спецификация	specification_id, version_id, дата создания	specification_id	version_id
Позиция спецификации	item_id, specification_id, material_id, количество	item_id	specification_id, material_id
Материал	material_id, material_code, наименование, category_code, единица измерения	material_id	category_code
Категория материала	category_code, наименование	category_code	-
Согласование	approval_id, version_id, approver_id, approval_status_code, комментарий, дата решения	approval_id	version_id, approver_id, approval_status_code
Статус согласования	approval_status_code, наименование	approval_status_code	-
Закупочная потребность	procurement_id, application_id, specification_id, procurement_status_code, дата создания, требуемая дата	procurement_id	application_id, specification_id, procurement_status_code
Статус закупочной потребности	procurement_status_code, наименование, описание	procurement_status_code	-
Позиция закупки	procurement_item_id, procurement_id, material_id, количество, требуемая дата	procurement_item_id	procurement_id, material_id
Журнал действий	log_id, application_id, user_id, действие, старое значение, новое значение, дата и время	log_id	application_id, user_id
Уведомление	notification_id, user_id, application_id, текст, статус, дата создания, дата прочтения	notification_id	user_id, application_id

Структура модели соответствует ранее спроектированному целевому процессу: после регистрации заявки назначается дизайнер, формируется версия концепции и спецификация, выполняется согласование, а после получения положительных решений создаётся закупочная потребность.

2. Основные связи между сущностями

Между сущностями определены следующие отношения:

- один пользователь в роли покупателя может иметь несколько объектов ремонта - 1:N;
- один объект ремонта может иметь несколько заявок - 1:N;
- одной заявке соответствует одна концепция - 1:1;
- концепция может содержать несколько версий - 1:N;
- каждая версия концепции может содержать несколько файлов - 1:N;
- одной версии соответствует спецификация - 1:1;
- спецификация содержит несколько позиций - 1:N;
- один материал может использоваться во множестве спецификаций - 1:N;
- одна версия концепции может иметь несколько результатов согласования - 1:N;
- одна заявка может иметь одну или несколько закупочных потребностей - 1:N;
- закупочная потребность содержит несколько позиций - 1:N;
- для одной заявки может быть создано множество записей журнала действий - 1:N;
- пользователь может получать множество уведомлений - 1:N.
- один статус закупочной потребности может соответствовать нескольким закупочным потребностям — 1:N.

3. ER-диаграмма базы данных

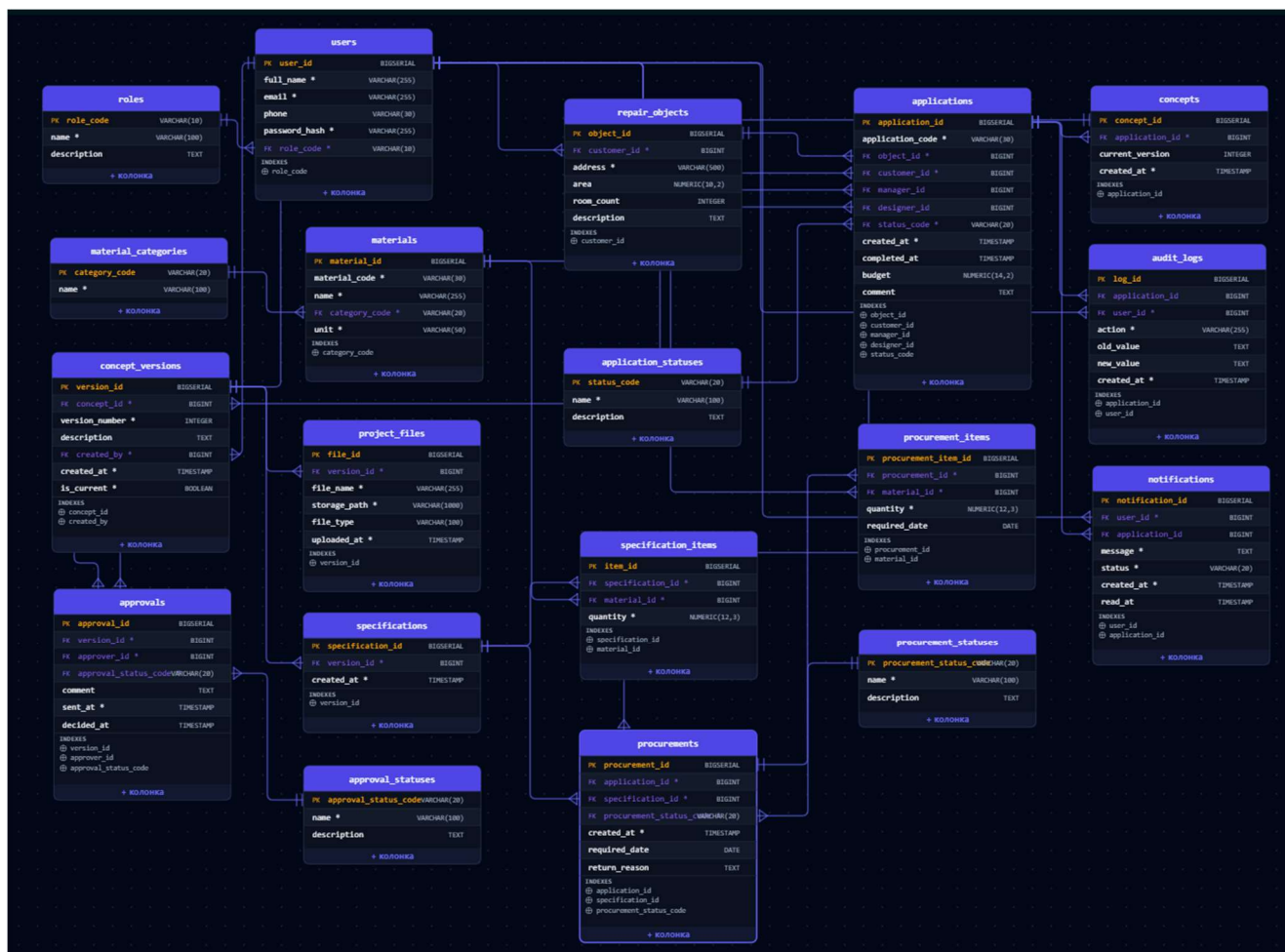


Рисунок 11 - Логическая модель базы данных проектируемой информационной системы

4. Разработка кодификаторов

Кодификаторы используются для исключения различных вариантов записи одного и того же значения и обеспечения единообразной обработки данных. Методические материалы прямо предусматривают применение кодификаторов для статусов, должностей и других справочных значений.

Таблица 7 - Кодификатор ролей пользователей

Код	Значение	Назначение
CUS	Покупатель	Создание и просмотр собственных заявок
MAN	Менеджер	Управление заявками и назначение исполнителей
DES	Дизайнер	Разработка концепции
PRO	Проектный отдел	Согласование проекта
CON	Отдел строительства	Согласование реализуемости
SUP	Снабжение	Работа с закупочной потребностью
ANA	Аналитик	Формирование аналитических отчётов
ADM	Администратор	Управление пользователями и справочниками

Таблица 8 - Кодификатор статусов заявки

Код	Статус	Описание
NEW	Новая	Заявка зарегистрирована
ASSIGNED	Назначена	Назначен ответственный дизайнер
DESIGN	Разработка концепции	Выполняется подготовка концепции
APPROVAL	На согласовании	Версия передана согласующим сторонам
REVISION	На доработке	Получены замечания
APPROVED	Согласована	Все необходимые решения положительны
PROCUREMENT	Передана в снабжение	Сформирована закупочная потребность
COMPLETED	Завершена	Работа по заявке завершена
CANCELLED	Отменена	Работа по заявке прекращена

Таблица 9 - Кодификатор результатов согласования

Для заявки		
Код	Значение	Описание
WAIT	Ожидает решения	Решение ещё не принято
APPROVED	Согласовано	Замечаний нет
REJECTED	Требуется доработка	Концепция возвращается дизайнеру
Для закупочной потребности		
CREATED	Сформирована	Закупочная потребность создана
ACCEPTED	Принята	Принята сотрудником снабжения
SENT	Передана поставщику	Данные отправлены поставщику
REJECTED	Возвращена	Возвращена с указанием причины
COMPLETED	Выполнена	Закупочная потребность закрыта

Таблица 10 - Кодификатор категорий материалов

Код	Значение
FIN	Отделочные материалы
FLOOR	Напольные покрытия
LIGHT	Освещение
SAN	Сантехника
FURN	Мебель
DECOR	Декор
OTHER	Прочее

5. Схема кодирования заявки

Для однозначной идентификации заявки используется код:

APP-YYYY-NNNN

где:

APP - постоянный префикс заявки;

YYYY - год регистрации;

NNNN - порядковый номер заявки в пределах года.

Например:

APP-2026-00125

означает заявку №125, зарегистрированную в 2026 году.

Такой подход соответствует принципу структурированного кодирования, описанному в методических материалах: код должен состоять из отдельных смысловых частей и иметь однозначные правила формирования.

Таблица 11 - Схема кодирования заявки

Наименование кода	Структура	Часть	Значение	Диапазон / пример	Примечание
Код заявки	APP-YYYY-NNNN	APP	Тип объекта «Заявка»	Постоянное значение	Определяет тип сущности
		YYYY	Год регистрации	2026	Четыре цифры
		NNNN	Порядковый номер	00001-99999	Автоматическая нумерация

Вывод

В результате практического занятия была разработана логическая модель базы данных информационной системы. Определены основные сущности, их атрибуты, первичные и внешние ключи, установлены отношения между сущностями. Для унификации данных разработаны кодификаторы ролей, статусов заявок, результатов согласования, статусов закупочных потребностей и категорий материалов, а также определена схема формирования идентификатора заявки.

Практическое занятие № 4

Цель занятия:

Разработать каркасные прототипы пользовательского интерфейса информационной системы, описать основные сценарии взаимодействия пользователей и построить диаграмму деятельности, отображающую логику работы системы.

По методическим указаниям результат должен включать wireframes, Use Case Scenarios, Activity Diagram и проверку согласованности интерфейсов.

Постановка задачи:

1. Определить основные пользовательские экраны системы.
2. Разработать каркасные прототипы интерфейсов.
3. Описать основные сценарии использования системы.
4. Построить диаграмму деятельности процесса сопровождения заявки.
5. Проверить соответствие интерфейсов функциональным требованиям системы.

Результат работы:

1. Разработка структуры пользовательского интерфейса

В проектируемой системе предусмотрены интерфейсы для покупателя и сотрудников организации. В предыдущем проектировании уже были определены основные сценарии: создание заявки, подготовка концепции, согласование, формирование закупочной потребности и получение аналитики.

Для системы разработаны следующие основные экраны.

Таблица 12 - Каркасные прототипы интерфейсов

Экран	Пользователь	Основные элементы
Авторизация	Все пользователи	Email/логин, пароль, кнопка «Войти»
Список заявок	Покупатель, сотрудники	Поиск, фильтр по статусу, номер заявки, объект, ответственный, дата
Создание заявки	Покупатель	Адрес объекта, площадь, планировка, стиль, материалы, бюджет, комментарий, загрузка файлов
Карточка заявки	Покупатель, сотрудники	Общая информация, текущий статус, ответственные, история
Концепция	Дизайнер	Описание концепции, файлы, версия, спецификация материалов
Согласование	Покупатель, проектный отдел, строительство	Просмотр файлов, «Согласовать», «Вернуть на доработку», комментарий
Снабжение	Сотрудник снабжения	Список материалов, количество, сроки, принятие или возврат заявки
Аналитика	Аналитик	Период, фильтры, показатели, таблицы и диаграммы
Администрирование	Администратор	Пользователи, роли, справочники, журнал действий

2. Каркасные прототипы

Экран «Список заявок»

Система ремонта

Заявки

Аналитика

АС Алексей Смирнов

Заявки

Аналитика

Справочники

Настройки

Заявки

Поиск

Статус

Ответственный

Период

+ Создать заявку

№ заявки	Объект	Покупатель	Статус	Ответственный	Дата
APP-2026-00125	ул. Примерная, 15	Иванов И.И.	На согласовании	Петров А.А.	10.09.2026
APP-2026-00124	ул. Центральная, 7	Смирнова Е.В.	На доработке	Сидоров Д.В.	09.09.2026
APP-2026-00123	пр-т Мира, 22	Кузнецов А.В.	Согласована	Петров А.А.	08.09.2026

Показано 3 из 125 заявок

Рисунок 12 - Экранная форма «Список заявок»

Экран «Создание заявки»

Система ремонта

Заявки

Аналитика

АС Алексей Смирнов

Заявки

Аналитика

Справочники

Настройки

Новая заявка

Адрес объекта *

Введите адрес

Желаемый стиль

Выберите стиль

Площадь

м²

Бюджет

Р

Количество комнат

Введите количество

Пожелания

Опишите пожелания к проекту

План / фотографии

Перетащите файл сюда или выберите на компьютере

plan.pdf room.jpg

Отмена

Создать заявку

Рисунок 13 - Экранная форма «Создание заявки»

Экран «Карточка заявки»

Система ремонта

Заявки

Аналитика

АС Алексей Смирнов

Заявки

Аналитика

Справочники

Настройки

APP-2026-00125

На согласовании

Общая информация

Концепция

Согласование

Снабжение

История

Общая информация

Объект
ул. Примерная, д. 15, кв. 42

Покупатель
Иванов Иван Иванович

Менеджер
Смирнов Алексей Сергеевич

Дизайнер
Петров Андрей Александрович

Бюджет
1 500 000 Р

Дата создания
10.09.2026

Текущая версия концепции

Версия №3

Статус: На согласовании

Открыть концепцию

Последние события

10.09.2026

Версия №3 направлена на согласование

09.09.2026

Дизайнер загрузил новую версию

08.09.2026

Версия №2 возвращена на доработку

Рисунок 14 - Экранная форма «Карточка заявки»

Экран «Согласование концепции»

Система ремонта

Заявки

Аналитика

АС Алексей Смирнов

Заявки

Аналитика

Справочники

Настройки

Согласование концепции

Версия №3

Файлы проекта

Планировка.pdf

PDF

Просмотреть

Визуализация.jpg

JPG

Просмотреть

Спецификация.xlsx

XLSX

Просмотреть

Решения участников

Покупатель

Согласовано

Проектный отдел

Ожидает решения

Отдел строительства

Согласовано

Комментарий

Комментарий к решению

Введите комментарий...

Вернуть на доработку

Согласовать

Рисунок 15 - Экранная форма «Согласование концепции»

3. Сценарии использования

Таблица 13 - UC-01 «Создание заявки»

Элемент	Описание
Название	Создание заявки
Акторы	Покупатель
Предусловия	Покупатель авторизован
Основной поток	1. Покупатель открывает форму новой заявки. 2. Заполняет сведения об объекте. 3. Указывает пожелания и бюджет. 4. Прикрепляет файлы. 5. Нажимает «Создать заявку». 6. Система проверяет обязательные поля. 7. Система создаёт уникальный номер и статус заявки.
Альтернативный поток	Если обязательное поле не заполнено, система выделяет ошибку и не позволяет отправить форму.
Результат	Новая заявка зарегистрирована в системе

Таблица 14 - UC-02 «Подготовка концепции»

Элемент	Описание
Название	Подготовка концепции
Акторы	Дизайнер
Предусловия	Дизайнер назначен исполнителем заявки
Основной поток	1. Дизайнер открывает заявку. 2. Просматривает пожелания покупателя. 3. Создаёт новую версию концепции. 4. Загружает проектные файлы. 5. Заполняет спецификацию материалов. 6. Сохраняет версию. 7. Направляет концепцию на согласование.
Альтернативный поток	При возврате концепции дизайнер создаёт новую версию с учётом замечаний.
Результат	Актуальная версия концепции передана на согласование

Таблица 15 - UC-03 «Согласование концепции»

Элемент	Описание
Название	Согласование концепции
Акторы	Покупатель, проектный отдел, отдел строительства
Предусловия	Существует актуальная версия концепции
Основной поток	1. Пользователь получает уведомление. 2. Открывает концепцию. 3. Просматривает материалы. 4. Выбирает «Согласовать». 5. Система сохраняет решение.
Альтернативный поток	Пользователь выбирает «Вернуть на доработку» и обязан указать комментарий.
Результат	Концепция согласована либо возвращена дизайнеру

Таблица 16 - UC-04 «Формирование закупочной потребности»

Элемент	Описание
Название	Формирование закупочной потребности
Акторы	Сотрудник снабжения
Предусловия	Все обязательные согласования имеют положительный результат
Основной поток	1. Система получает утверждённую спецификацию. 2. Формирует перечень необходимых материалов. 3. Рассчитывает количества. 4. Создаёт закупочную потребность. 5. Снабжение принимает её в работу.
Альтернативный поток	Снабжение возвращает заявку с указанием причины.
Результат	Закупочная потребность принята в работу

4. Диаграмма деятельности

Логика Activity Diagram строится вокруг ранее определённого TO-BE процесса: регистрация заявки → назначение дизайнера → разработка концепции → согласование → при необходимости доработка → передача в снабжение.

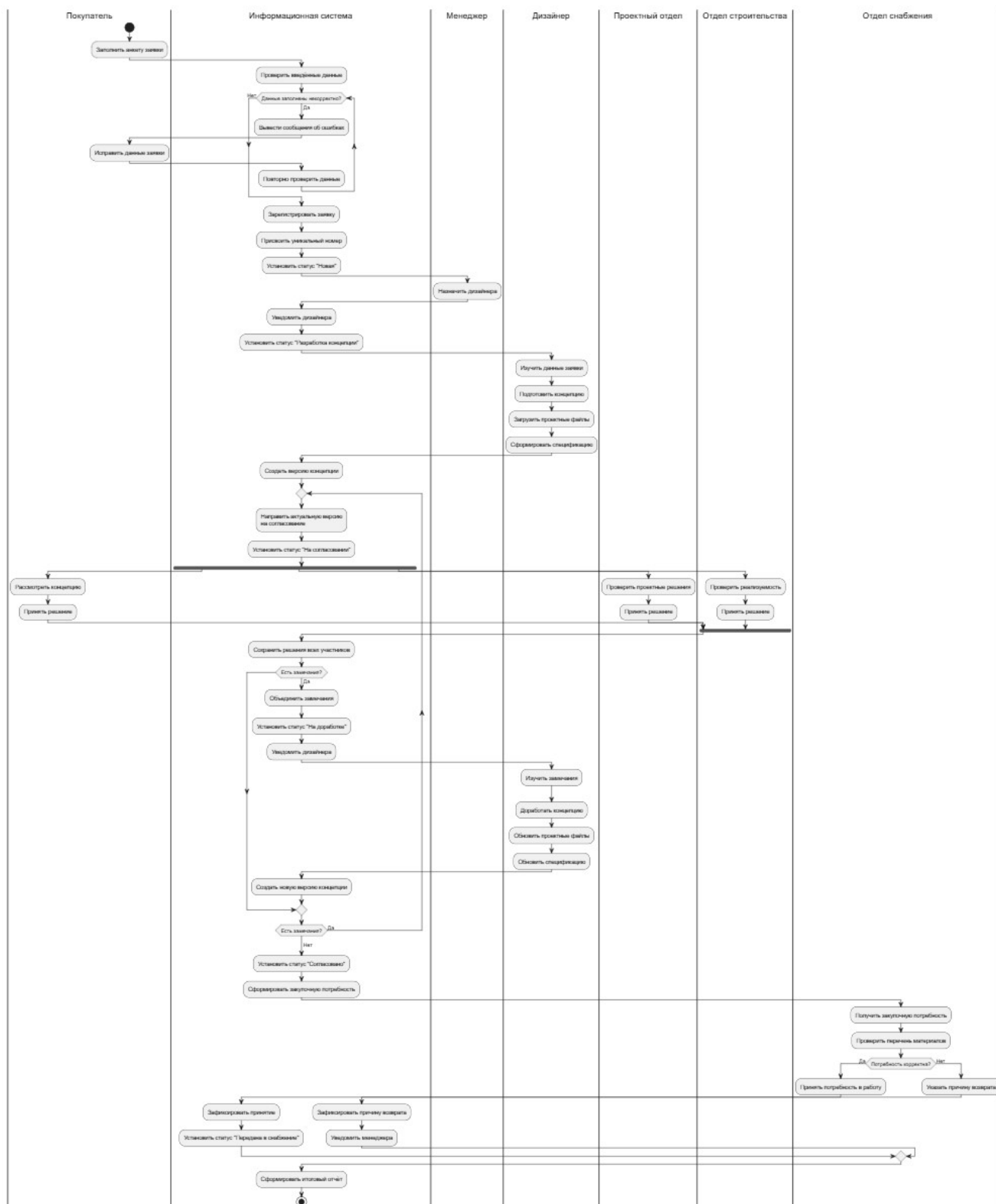


Рисунок 16 - Диаграмма деятельности процесса сопровождения заявки

5. Проверка согласованности интерфейсов

Разработанные интерфейсы соответствуют пользовательским сценариям и ранее сформированным функциональным требованиям.

Форма создания заявки обеспечивает выполнение FR-01 и FR-02. Карточка заявки и список заявок реализуют отображение статусов и поиск. Экран концепции обеспечивает создание и хранение версий. Интерфейс согласования реализует принятие решений и возврат концепции с обязательным комментарием. Экран снабжения обеспечивает работу с закупочной потребностью, а аналитический раздел - формирование статистики по завершённым проектам. Требования к данным функциям ранее были определены в практическом занятии №1.

Вывод

В результате практического занятия разработана структура пользовательских интерфейсов информационной системы, подготовлены каркасные прототипы основных экранов, детализированы сценарии взаимодействия пользователей и построена диаграмма деятельности. Разработанные интерфейсы обеспечивают выполнение основных функций проектируемой системы и поддерживают сквозной процесс сопровождения заявки.

Практическое занятие № 5

Цель занятия:

Сформировать математическое обеспечение проектируемой информационной системы, определить рассчитываемые показатели, установить связи между ними и разработать алгоритмы обработки данных.

Методические материалы требуют определить показатели предметной области, установить их взаимосвязи, сформировать расчётные формулы и разработать блок-схемы алгоритмов.

Постановка задачи:

1. Определить показатели, необходимые для анализа работы системы.
2. Определить исходные данные для каждого показателя.
3. Разработать математические формулы расчёта.
4. Определить взаимосвязи показателей.
5. Разработать алгоритмы обработки данных.
6. Построить блок-схемы расчётов.

Результат работы:

1. Определение аналитических показателей

В предыдущих практических занятиях для системы предусмотрена функция аналитики, включающая сбор фактических данных, расчёт популярности материалов, фильтрацию и формирование отчётов.

Для оценки эффективности процесса сопровождения заявки были выбраны следующие показатели:

1. среднее время обработки заявки;
2. среднее время согласования концепции;
3. среднее количество доработок концепции;
4. доля концепций, согласованных с первой попытки;
5. популярность материала;
6. доля закупочных потребностей, принятых в работу.

2. Формулы расчёта

Среднее время обработки заявки

Среднее время от регистрации до завершения заявки:

$$T_{avg} = \frac{\sum_{i=1}^N (t_{finis\ i} - t_{start_i})}{N}$$

где:

- N - количество завершённых заявок;
- t_{start_i} - дата регистрации заявки;
- $t_{finis\ i}$ - дата её завершения.

Единица измерения - часы или дни.

Среднее время согласования

$$T_{approval} = \frac{\sum_{i=1}^M (t_{decision_i} - t_{send_i})}{M}$$

где:

- M - количество выполненных согласований;
- t_{send_i} - момент отправки на согласование;
- $t_{decision_i}$ - момент принятия решения.

Среднее количество доработок

$$R_{avg} = \frac{\sum_{i=1}^N R_i}{N}$$

где:

- R_i - количество возвратов концепции на доработку для заявки;
- N - количество завершённых заявок.

Доля согласований без доработки

$$P_{first} = \frac{N_{first}}{N_{approved}} \times 100\%$$

где:

- N_{first} - число концепций, согласованных без возврата;
- $N_{approved}$ - общее число согласованных концепций.

Популярность материала

Для определения популярности материала используется доля завершённых проектов, в которых материал фактически применялся:

$$P_{material_j} = \frac{N_{material_j}}{N_{projects}} \times 100\%$$

где:

- $N_{material_j}$ - количество завершённых проектов, в которых использован материал j ;
- $N_{projects}$ - общее количество завершённых проектов за выбранный период.

Именно расчёт популярности товаров по завершённым проектам был предусмотрен функциональным требованием FR-11.

Доля принятых закупочных потребностей

$$P_{procurement} = \frac{N_{accepted}}{N_{created}} \times 100\%$$

где:

- $N_{accepted}$ - количество потребностей, принятых снабжением;
- $N_{created}$ - общее число сформированных закупочных потребностей.

3. Таблица показателей и взаимосвязей

Таблица 17 - Аналитические показатели системы

Показатель	Формула	Входные данные	Результат
Среднее время обработки заявки	$T_{avg} = \frac{\sum(t_{finish} - t_{start})}{N}$	Дата создания, дата завершения	Среднее время, дни
Среднее время согласования	$T_{approval} = \frac{\sum(t_{decision} - t_{send})}{M}$	Время отправки и принятия решения	Среднее время
Среднее количество доработок	$R_{avg} = \frac{\sum R_i}{N}$	Количество возвратов по заявкам	Среднее число
Согласование с первой попытки	$P_{first} = \frac{N_{first}}{N_{approved}} \times 100\%$	Версии концепций и результаты согласований	%
Популярность материала	$P_j = \frac{N_j}{N} \times 100\%$	Завершённые проекты и использованные материалы	%
Приём закупочных потребностей	$P_{proc} = \frac{N_{accepted}}{N_{created}} \times 100\%$	Статусы закупочных потребностей	%

4. Взаимосвязь показателей

Показатели обработки заявки связаны между собой. Увеличение количества возвратов концепции на доработку приводит к увеличению общего времени выполнения заявки и времени согласования. Снижение числа доработок, напротив, повышает долю заявок, согласованных с первой попытки.

Общую продолжительность сопровождения заявки можно представить следующим образом:

$$T_{total} = T_{registration} + T_{design} + T_{approval} + T_{revision} + T_{procurement}$$

где:

- $T_{registration}$ - время регистрации и назначения исполнителя;
- T_{design} - разработка концепции;
- $T_{approval}$ - согласование;
- $T_{revision}$ - время доработок;
- $T_{procurement}$ - подготовка и передача закупочной потребности.

При отсутствии возвратов:

$$T_{revision} = 0$$

Следовательно, сокращение количества циклов доработки непосредственно уменьшает общую продолжительность обработки заявки.

5. Алгоритм расчёта среднего времени обработки заявки

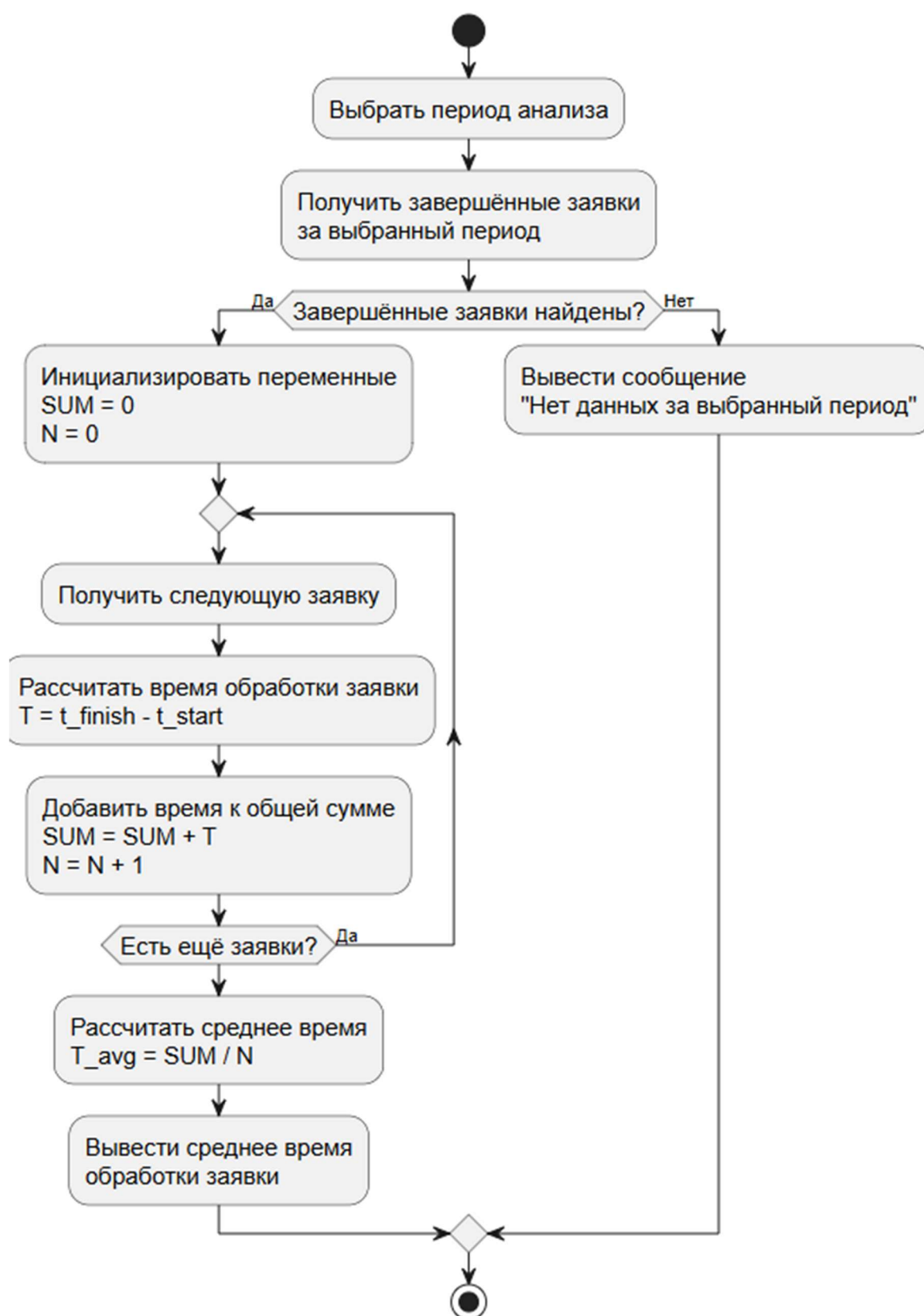


Рисунок 17 - Алгоритм расчёта среднего времени обработки заявки

6. Алгоритм расчёта популярности материалов

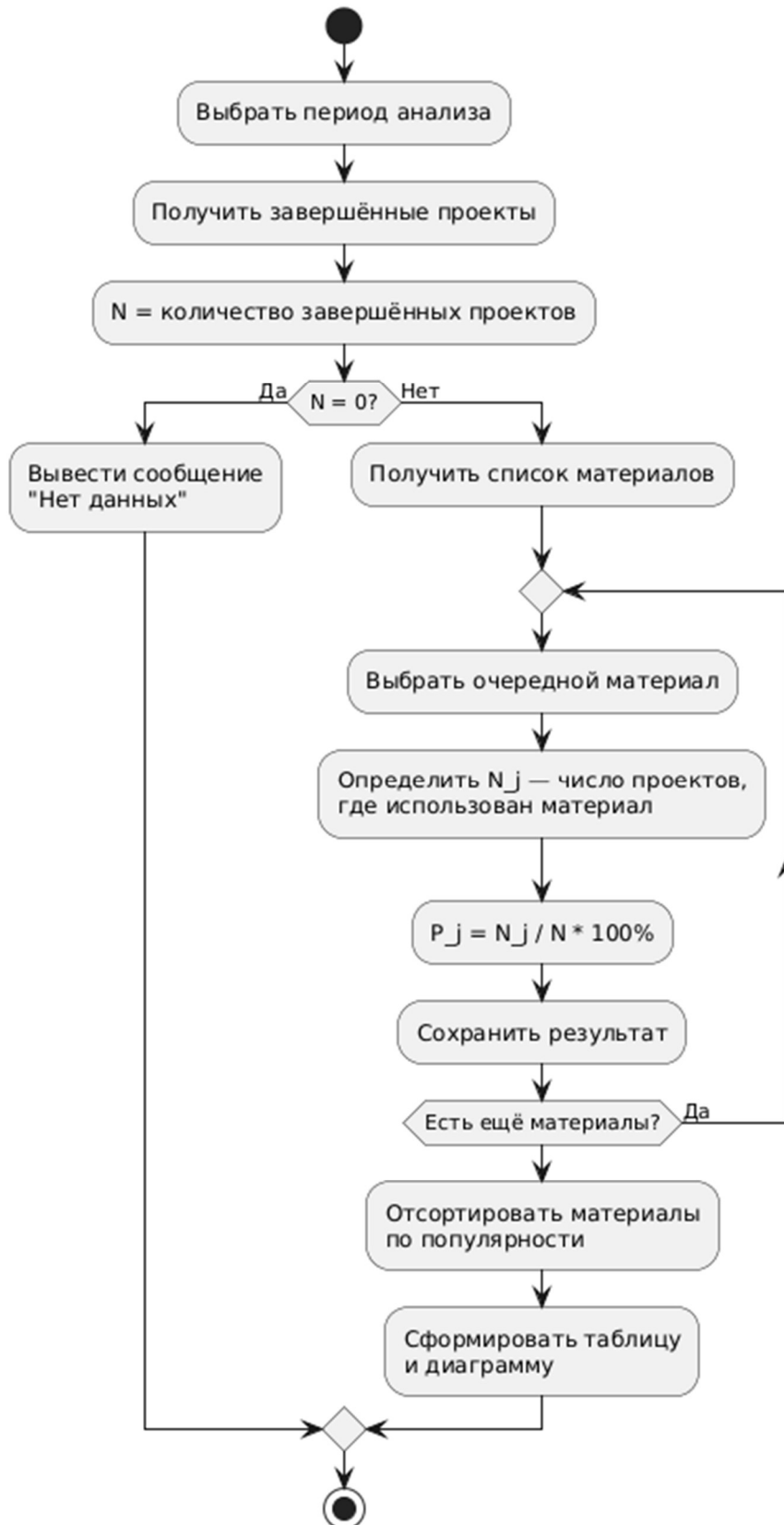


Рисунок 18 - Алгоритм расчёта популярности материалов

7. Алгоритм определения доли согласований с первой попытки

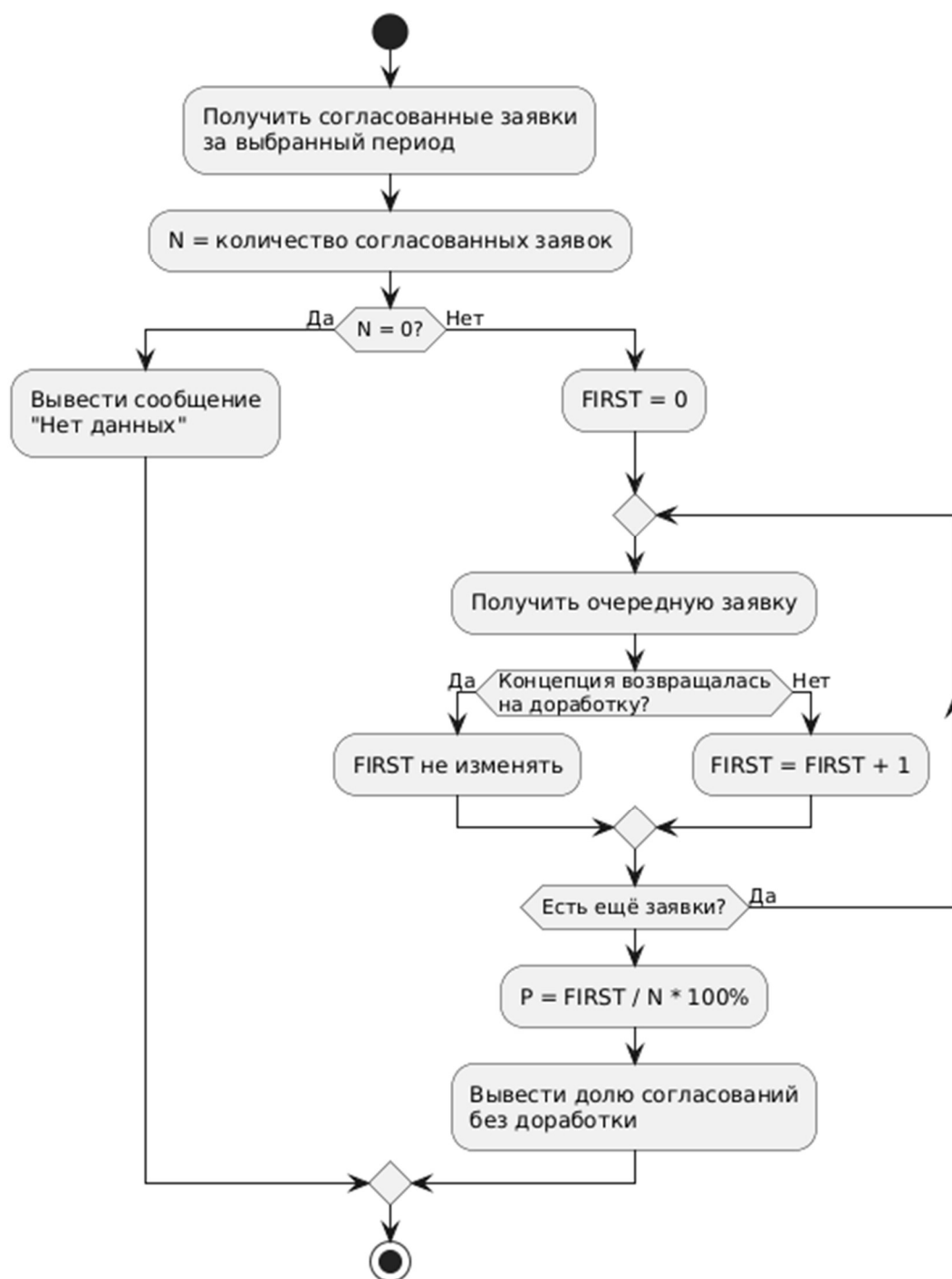


Рисунок 19 - Алгоритм расчёта доли концепций, согласованных без доработки

8. Использование показателей в информационной системе

Рассчитанные показатели отображаются в аналитическом разделе системы. Пользователь выбирает период анализа, после чего система получает данные завершённых заявок, выполняет расчёты и формирует таблицы и диаграммы.

Например, аналитическая панель может содержать:

Показатель	Значение
Завершено заявок	124
Среднее время обработки	18,4 дня
Среднее время согласования	2,8 дня
Среднее количество доработок	1,6
Согласовано с первой попытки	42 %
Закупочных потребностей принято	91 %

Приведённые значения являются демонстрационными и используются для иллюстрации представления аналитических показателей.

Вывод

В результате практического занятия сформировано математическое обеспечение информационной системы. Определены основные показатели эффективности процесса сопровождения заявки, разработаны формулы расчёта среднего времени обработки и согласования, количества доработок, доли успешных согласований, популярности материалов и эффективности формирования закупочной потребности. Для ключевых показателей разработаны алгоритмы обработки данных и блок-схемы. Это обеспечивает возможность последующего создания аналитического модуля системы.

Практическое занятие № 6

Цель занятия:

Определить архитектуру инфраструктуры проектируемой информационной системы, выбрать среду разработки и систему управления базами данных.

Постановка задачи:

1. Построить инфраструктурную схему, отражающую состав и взаимодействие технических средств информационной системы.
2. Выполнить сравнительный анализ программных средств разработки.
3. Выполнить сравнительный анализ систем управления базами данных.
4. Выбрать оптимальную среду разработки и СУБД для реализации проектируемой информационной системы.
5. Обосновать принятые решения.

Результат работы:

1. Проектирование инфраструктуры информационной системы

Ранее для проектируемой системы была определена компонентная архитектура, в которой web-интерфейсы обеспечивают доступ пользователей по HTTPS, сервис заявок обрабатывает карточки, версии и статусы, сервис согласований управляет решениями участников, структурированные данные и журнал аудита хранятся в базе данных, а файлы концепций - в отдельном файловом хранилище.

С учётом данной архитектуры предлагается использовать клиент-серверную инфраструктуру. Пользователи работают с системой через браузер со своих рабочих станций. Запросы передаются по сети Интернет с использованием HTTPS и проходят через межсетевой экран. Далее запрос поступает на web-сервер, который перенаправляет его серверу приложений.

Сервер приложений реализует бизнес-логику системы: регистрацию заявок, работу с концепциями и версиями, согласование, формирование закупочной потребности, аналитику и управление правами доступа.

Для хранения структурированных данных используется отдельный сервер PostgreSQL. Файлы проектов и концепций размещаются в файловом хранилище. Для обеспечения восстановления информации предусматривается резервное хранилище.

Такое разделение соответствует уже установленным требованиям проекта: полная резервная копия базы данных должна создаваться не реже одного раза в сутки, а система должна поддерживать хранение не менее 100 000 заявок.

2. Состав инфраструктуры

Таблица 18 - Состав технической инфраструктуры информационной системы

Компонент	Назначение
Рабочая станция покупателя	Работа с заявками через web-интерфейс
Рабочая станция сотрудника	Работа менеджера, дизайнера, согласующих подразделений, снабжения и администратора
Интернет	Передача данных между клиентами и серверной инфраструктурой
Маршрутизатор / межсетевой экран	Контроль сетевого доступа и защита серверного сегмента
Web-сервер	Приём HTTPS-запросов и передача их серверу приложений
Сервер приложений	Выполнение бизнес-логики информационной системы
Сервер СУБД	Хранение заявок, пользователей, статусов, спецификаций, согласований и журнала действий
Файловое хранилище	Хранение планировок, изображений, концепций и других файлов проекта
Сервис уведомлений	Формирование и передача уведомлений пользователям
Резервное хранилище	Хранение резервных копий базы данных и критически важных файлов

3. Инфраструктурная схема

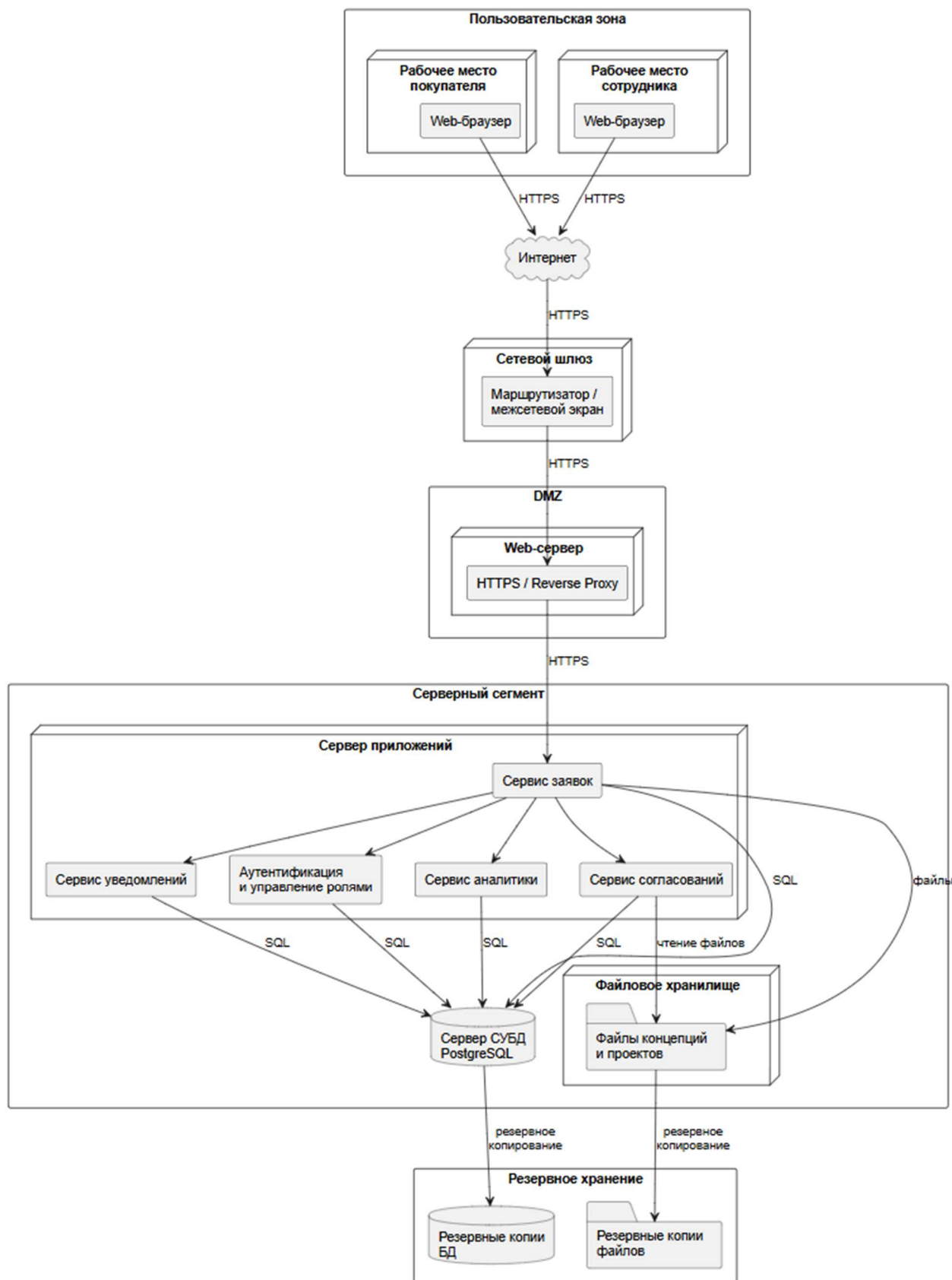


Рисунок 20 - Инфраструктурная схема проектируемой информационной системы

4. Сравнительный анализ сред разработки

Для реализации серверной части проектируемой информационной системы рассматриваются среды разработки, поддерживающие язык **Java** и современные средства создания web-приложений.

Таблица 19 - Сравнение сред разработки

Критерий	IntelliJ IDEA	Eclipse	Visual Studio Code
Стоимость	Community Edition - бесплатно, расширенная версия - платная	Бесплатная	Бесплатная
Поддержка Java	Очень высокая	Очень высокая	Высокая при установке расширений
Поддержка Spring	Высокая	Высокая	Поддерживается через расширения
Поддержка Maven/Gradle	Встроенная	Встроенная	Через расширения
Средства отладки	Развитые	Развитые	Хорошие
Рефакторинг кода	Очень развитый	Развитый	Более ограниченный
Работа с Git	Встроенная	Поддерживается	Встроенная
Расширяемость	Высокая	Высокая	Очень высокая
Удобство использования	Очень высокое	Высокое	Высокое
Требования к ресурсам	Высокие	Средние	Низкие
Поддержка и сообщество	Очень высокая	Высокая	Очень высокая
Пригодность для проекта	Очень высокая	Высокая	Высокая

Выбор среды разработки

В качестве основной среды разработки выбрана **IntelliJ IDEA**.

Выбор обусловлен развитой поддержкой языка Java, средствами автоматического рефакторинга, навигации по проекту, отладки и тестирования, а также интеграцией с системами сборки **Maven** и **Gradle**.

Среда хорошо подходит для разработки серверных web-приложений и позволяет организовать работу с многоуровневой структурой проекта, содержащей контроллеры, сервисы, сущности предметной области и репозитории доступа к данным.

Для проектируемой информационной системы это особенно важно, поскольку ранее были выделены отдельные функциональные области: управление заявками, концепциями и версиями, согласование, закупочная потребность, аналитика, уведомления и управление доступом.

5. Выбор технологии серверной части

В качестве языка программирования серверной части выбран Java, а в качестве основного web-фреймворка - **Spring Boot**.

Spring Boot позволяет реализовать серверное web-приложение и REST API, обеспечивающее взаимодействие клиентской части с бизнес-логикой и базой данных.

В проектируемой системе **Spring Boot** будет использоваться для реализации следующих функциональных модулей:

- управление пользователями и ролями;
- регистрация и сопровождение заявок;
- работа с объектами ремонта;
- управление концепциями и их версиями;
- хранение сведений о проектных файлах;
- согласование концепции;
- ведение спецификаций материалов;
- формирование закупочной потребности;
- уведомления пользователей;
- формирование аналитических данных;
- ведение журнала действий.

Для организации доступа к реляционной базе данных предлагается использовать **Spring Data JPA** и ORM-фреймворк **Hibernate**.

Общая логическая структура серверного приложения имеет следующий вид:

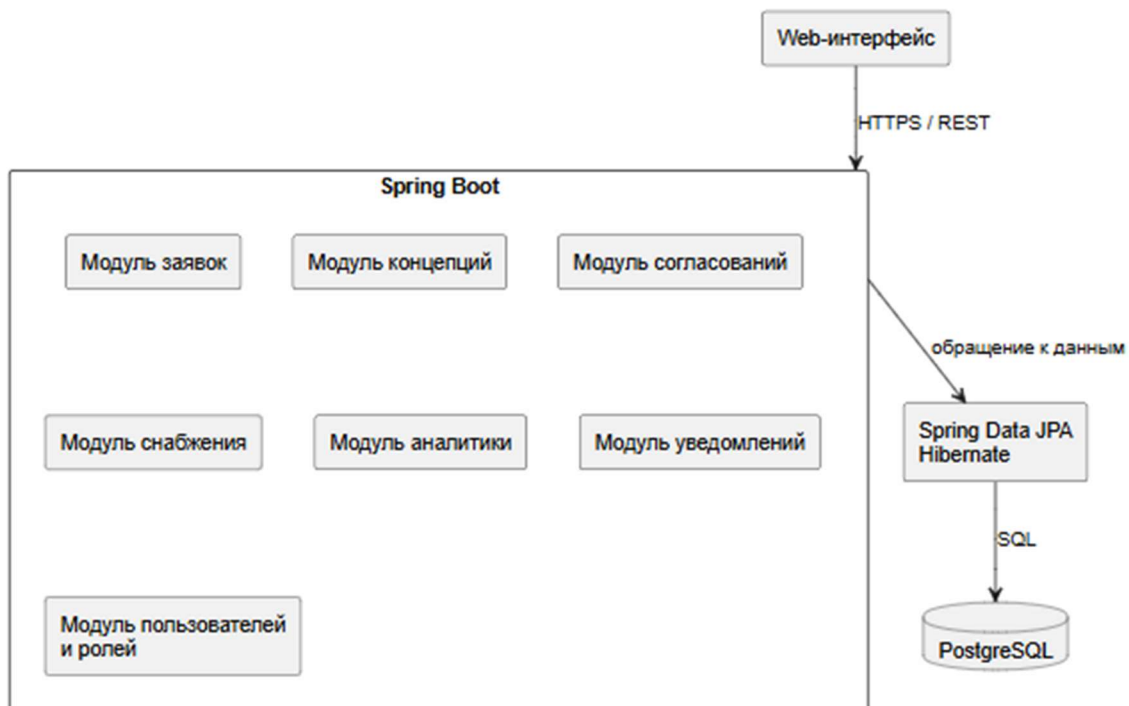


Рисунок 21 - Логическая структура серверного приложения

Взаимодействие серверного приложения с базой данных осуществляется через Spring Data JPA. ORM-фреймворк Hibernate выполняет отображение Java-объектов на таблицы PostgreSQL и преобразует операции над объектами в SQL-запросы.

6. Сравнительный анализ СУБД

В методических материалах в качестве примера рассматривается сравнение MySQL, PostgreSQL, Microsoft SQL Server и Oracle.

Таблица 20 - Сравнительный анализ СУБД

Критерий	PostgreSQL	MySQL	MS SQL Server	Oracle Database
Стоимость	Бесплатная	Бесплатная	Есть бесплатная и платные редакции	Преимущественно платная
Реляционная модель	Да	Да	Да	Да
Поддержка транзакций	Да	Да	Да	Да
Внешние ключи и ограничения	Да	Да	Да	Да
Работа с JSON	Развитая	Да	Да	Да
Производительность	Высокая	Высокая	Высокая	Очень высокая
Расширяемость	Очень высокая	Высокая	Высокая	Высокая
Масштабируемость	Высокая	Высокая	Высокая	Очень высокая
Совместимость с Java	Очень высокая	Очень высокая	Высокая	Высокая
Поддержка Hibernate/JPA	Да	Да	Да	Да
Администрирование	Средней сложности	Относительно простое	Средней сложности	Сложное
Поддержка сообщества	Очень высокая	Очень высокая	Высокая	Высокая
Пригодность для проекта	Очень высокая	Высокая	Высокая	Избыточная

7. Выбор системы управления базами данных

В качестве СУБД для проектируемой информационной системы выбрана PostgreSQL.

Основными причинами выбора являются:

- отсутствие необходимости приобретения коммерческой лицензии;
- развитая поддержка реляционной модели данных;
- обеспечение ссылочной целостности;
- поддержка транзакций;
- развитая система индексов;
- возможность работы со сложными запросами;
- поддержка JSON при необходимости хранения дополнительных структурированных данных;
- высокая совместимость с Java, Hibernate и Spring Data JPA;
- возможность дальнейшего масштабирования.

Выбор реляционной СУБД соответствует структуре проектируемой системы. В ней присутствует большое количество связанных сущностей: пользователи, роли, объекты ремонта, заявки, концепции, версии, спецификации, материалы, согласования, закупочные потребности и журнал действий.

Кроме того, ранее для системы было установлено требование поддерживать хранение не менее 100 000 заявок, а также выполнять регулярное резервное копирование базы данных.

Таким образом, PostgreSQL обеспечивает необходимые средства хранения, контроля целостности и обработки данных проектируемой системы.

8. Выбранный программный стек

По результатам сравнительного анализа сформирован программный стек проектируемой информационной системы.

Таблица 21 - Выбранные программные средства

Уровень	Средство	Назначение
Среда разработки	IntelliJ IDEA	Разработка, отладка и тестирование приложения
Язык программирования	Java	Реализация серверной бизнес-логики
Web-фреймворк	Spring Boot	Создание серверного приложения и REST API
Работа с данными	Spring Data JPA	Организация доступа к базе данных
ORM	Hibernate	Отображение Java-объектов на таблицы БД
Безопасность	Spring Security	Аутентификация и разграничение прав пользователей
СУБД	PostgreSQL	Хранение структурированных данных
Система сборки	Maven	Управление зависимостями и сборка приложения
Web-сервер / Reverse Proxy	Nginx	Обработка HTTPS и перенаправление запросов
Файловое хранилище	Выделенное файловое хранилище	Хранение файлов концепций и проектов
Клиент	Web-браузер	Работа пользователей с системой
Протокол взаимодействия	HTTPS / REST	Защищённый обмен данными

9. Обоснование выбранной архитектуры

Для серверной части информационной системы используется приложение на **Java с применением Spring Boot**.

Клиенты работают с системой через web-интерфейс. Запросы передаются по протоколу HTTPS через Nginx к Spring Boot-приложению.

Spring Boot реализует основные функции системы и предоставляет REST API клиентской части.

Для разграничения пользовательских прав используется Spring Security. Это соответствует ранее сформулированным требованиям системы: внутренние пользователи должны работать под индивидуальными учётными записями, а доступ к операциям должен зависеть от назначенных прав.

С точки зрения программной структуры серверное приложение предлагается разделить на следующие уровни:

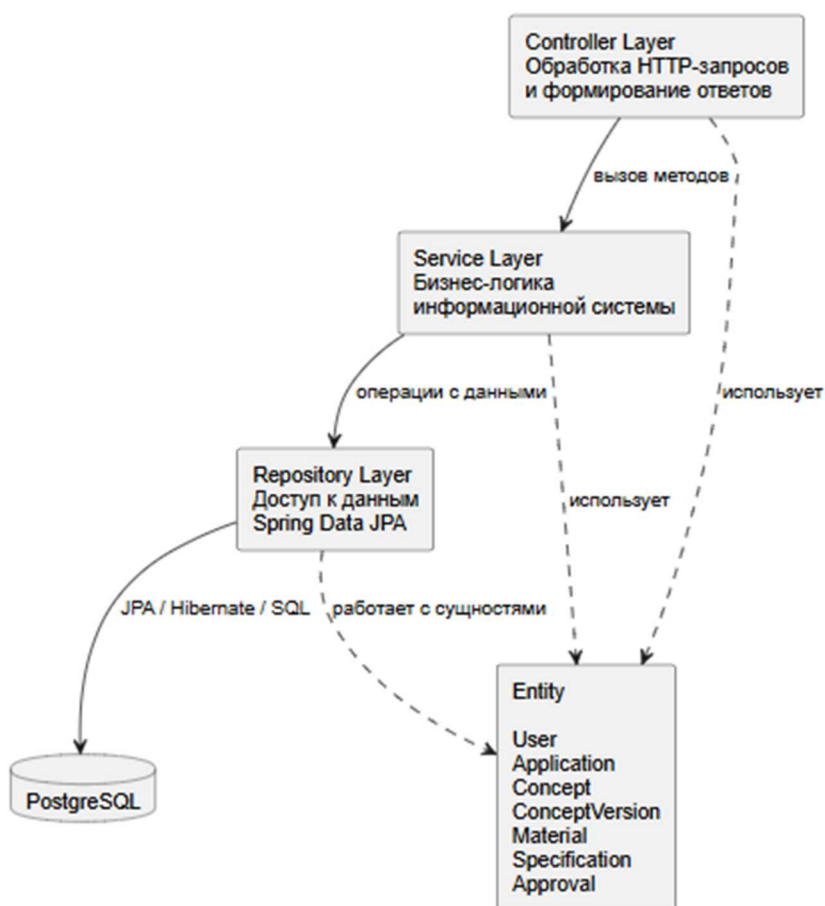


Рисунок 22 - Уровневая структура серверного приложения

Controller отвечает за получение HTTP-запросов и формирование ответов.

Service содержит бизнес-логику системы: изменение статусов, создание версий концепций, согласование, формирование закупочной потребности и расчёт аналитических показателей.

Repository обеспечивает доступ к данным через Spring Data JPA.

Entity представляет сущности предметной области, соответствующие таблицам базы данных.

Таблица 22 - Пример структуры серверного проекта

Пакет	Основные классы
controller	ApplicationController, ConceptController, ApprovalController, ProcurementController
service	ApplicationService, ConceptService, ApprovalService, ProcurementService, AnalyticsService
repository	ApplicationRepository, ConceptRepository, ApprovalRepository, MaterialRepository
entity	User, Application, Concept, ConceptVersion, Material, Specification, Approval
security	SecurityConfig
корневой пакет	RenovationApplication

Такое разделение повышает сопровождаемость приложения и позволяет изменять отдельные части системы без необходимости переработки всей кодовой базы.

10. Соответствие выбранного стека требованиям системы

Выбранный программный стек позволяет реализовать требования, определённые в предыдущих практических занятиях.

Например:

Требование	Средство реализации
Регистрация заявки	Spring Boot, Spring MVC
Управление статусами	Service Layer
Хранение версий концепций	PostgreSQL, JPA
Загрузка и хранение файлов	Spring Boot + файловое хранилище
Согласование концепций	Spring Boot
Разграничение прав	Spring Security
Работа с пользователями и ролями	Spring Security + PostgreSQL
Закупочная потребность	Service Layer + PostgreSQL
Аналитические запросы	PostgreSQL + Spring Data JPA
Журнал действий	PostgreSQL
REST API	Spring Boot
Резервное копирование	PostgreSQL + инфраструктурные средства

Функциональные требования исходного проекта уже предусматривают регистрацию заявок, версии концепций, согласование, закупочную потребность, аналитику и управление ролями.

Вывод

В ходе выполнения практического занятия была разработана инфраструктурная архитектура информационной системы и проведён сравнительный анализ программных средств разработки и систем управления базами данных.

В качестве основной среды разработки выбрана **IntelliJ IDEA**, в качестве языка программирования - **Java**, а серверная часть системы будет реализована с использованием **Spring Boot**.

Для взаимодействия приложения с базой данных выбраны **Spring Data JPA** и **Hibernate**, а в качестве системы управления базами данных - **PostgreSQL**.

Для управления зависимостями и сборки проекта предлагается использовать **Maven**, для реализации аутентификации и авторизации - **Spring Security**, а для обработки внешних **HTTPS-запросов** - **Nginx**.

Выбранный стек позволяет реализовать функциональные требования информационной системы, обеспечивает разделение бизнес-логики, доступа к данным и пользовательского интерфейса, а также допускает дальнейшее расширение и масштабирование приложения.

Практическое занятие № 7

Цель занятия:

Разработать физическую модель данных проектируемой информационной системы и сформировать диаграмму развертывания, отражающую размещение программных компонентов на вычислительных узлах.

Постановка задачи:

1. Разработать физическую модель базы данных с указанием таблиц, типов данных, размеров и ограничений.
2. Составить табличное описание атрибутов таблиц.
3. Определить первичные и внешние ключи, ограничения NOT NULL, UNIQUE, CHECK, DEFAULT.
4. Определить необходимые индексы.
5. Построить Deployment Diagram проектируемой информационной системы.
6. Проверить соответствие физической модели ранее разработанной логической модели данных.

Результат работы:

1. Разработка физической модели базы данных

В практическом занятии №3 была сформирована логическая модель базы данных, включающая пользователей, роли, объекты ремонта, заявки, концепции и их версии, проектные файлы, спецификации, материалы, согласования, закупочные потребности, журнал действий и уведомления.

В практическом занятии №6 в качестве СУБД выбрана **PostgreSQL**, а доступ серверного приложения к данным предполагается осуществлять через Spring Data JPA и Hibernate.

При переходе к физической модели для PostgreSQL используются следующие основные типы:

Тип PostgreSQL	Назначение
BIGSERIAL	Автоматически генерируемые идентификаторы
BIGINT	Внешние ключи
VARCHAR(n)	Строки ограниченной длины
TEXT	Текст произвольной длины
NUMERIC(p,s)	Денежные значения и количества
INTEGER	Целочисленные показатели
BOOLEAN	Логические признаки
DATE	Календарные даты
TIMESTAMP	Дата и время

2. Таблицы физической модели базы данных

Список таблиц модели:

1. roles
2. users
3. repair_objects
4. application_statuses
5. applications
6. concepts
7. concept_versions
8. project_files
9. material_categories
10. materials
11. specifications
12. specification_items
13. approval_statuses
14. approvals
15. procurement_statuses
16. procurements
17. procurement_items
18. audit_logs
19. notifications

3. Описание атрибутов таблиц

Таблица 23 - Атрибуты таблицы *roles*

Атрибут	Тип данных	Размерность	Ограничения
role_code	VARCHAR	10	PK, NOT NULL
name	VARCHAR	100	NOT NULL, UNIQUE
description	TEXT	-	NULL

Таблица 24 - Атрибуты таблицы *users*

Атрибут	Тип данных	Размерность	Ограничения
user_id	BIGSERIAL	-	PK
full_name	VARCHAR	255	NOT NULL
email	VARCHAR	255	NOT NULL, UNIQUE
phone	VARCHAR	30	NULL
password_hash	VARCHAR	255	NOT NULL
role_code	VARCHAR	10	NOT NULL, FK → roles(role_code)
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
is_active	BOOLEAN	-	NOT NULL, DEFAULT TRUE

Таблица 25 - Атрибуты таблицы *repair_objects*

Атрибут	Тип данных	Размерность	Ограничения
object_id	BIGSERIAL	-	PK
customer_id	BIGINT	-	NOT NULL, FK → users(user_id)
address	VARCHAR	500	NOT NULL
area	NUMERIC	(10,2)	CHECK (area > 0)
room_count	INTEGER	-	CHECK (room_count > 0)
description	TEXT	-	NULL

Таблица 26 - Атрибуты таблицы *application_statuses*

Атрибут	Тип данных	Размерность	Ограничения
status_code	VARCHAR	20	PK
name	VARCHAR	100	NOT NULL, UNIQUE
description	TEXT	-	NULL

Таблица 27 - Атрибуты таблицы *applications*

Атрибут	Тип данных	Размерность	Ограничения
application_id	BIGSERIAL	-	PK
application_code	VARCHAR	30	NOT NULL, UNIQUE
object_id	BIGINT	-	NOT NULL, FK → repair_objects(object_id)
customer_id	BIGINT	-	NOT NULL, FK → users(user_id)
manager_id	BIGINT	-	FK → users(user_id)
designer_id	BIGINT	-	FK → users(user_id)
status_code	VARCHAR	20	NOT NULL, FK → application_statuses(status_code)
budget	NUMERIC	(14,2)	CHECK (budget >= 0)
comment	TEXT	-	NULL
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
completed_at	TIMESTAMP	-	NULL

Таблица 28 - Атрибуты таблицы *concepts*

Атрибут	Тип данных	Размерность	Ограничения
concept_id	BIGSERIAL	-	PK
application_id	BIGINT	-	NOT NULL, UNIQUE, FK → applications(application_id)
current_version	INTEGER	-	NOT NULL, DEFAULT 1, CHECK (current_version > 0)
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Таблица 29 - Атрибуты таблицы *concept_versions*

Атрибут	Тип данных	Размерность	Ограничения
version_id	BIGSERIAL	-	PK
concept_id	BIGINT	-	NOT NULL, FK → concepts(concept_id)
version_number	INTEGER	-	NOT NULL, CHECK (version_number > 0)
description	TEXT	-	NULL
created_by	BIGINT	-	NOT NULL, FK → users(user_id)
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
is_current	BOOLEAN	-	NOT NULL, DEFAULT TRUE

Для таблицы *concept_versions* устанавливается ограничение *UNIQUE* (*concept_id*, *version_number*).

Таблица 30 - Атрибуты таблицы *project_files*

Атрибут	Тип данных	Размерность	Ограничения
file_id	BIGSERIAL	-	PK
version_id	BIGINT	-	NOT NULL, FK → concept_versions(version_id)
file_name	VARCHAR	255	NOT NULL
storage_path	VARCHAR	1000	NOT NULL
file_type	VARCHAR	100	NULL
uploaded_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Таблица 31 - Атрибуты таблицы *material_categories*

Атрибут	Тип данных	Размерность	Ограничения
category_code	VARCHAR	20	PK
name	VARCHAR	100	NOT NULL, UNIQUE

Таблица 32 - Атрибуты таблицы *materials*

Атрибут	Тип данных	Размерность	Ограничения
material_id	BIGSERIAL	-	PK
material_code	VARCHAR	30	NOT NULL, UNIQUE
name	VARCHAR	255	NOT NULL
category_code	VARCHAR	20	NOT NULL, FK → material_categories(category_code)
unit	VARCHAR	50	NOT NULL
is_active	BOOLEAN	-	NOT NULL, DEFAULT TRUE

Таблица 33 - Атрибуты таблицы *specifications*

Атрибут	Тип данных	Размерность	Ограничения
specification_id	BIGSERIAL	-	PK
version_id	BIGINT	-	NOT NULL, UNIQUE, FK → concept_versions(version_id)
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Таблица 34 - Атрибуты таблицы *specification_items*

Атрибут	Тип данных	Размерность	Ограничения
item_id	BIGSERIAL	-	PK
specification_id	BIGINT	-	NOT NULL, FK → specifications(specification_id)
material_id	BIGINT	-	NOT NULL, FK → materials(material_id)
quantity	NUMERIC	(12,3)	NOT NULL, CHECK (quantity > 0)

Для таблицы *specification_items* устанавливается ограничение *UNIQUE* (*specification_id*, *material_id*).

Таблица 35 - Атрибуты таблицы *approval_statuses*

Атрибут	Тип данных	Размерность	Ограничения
approval_status_code	VARCHAR	20	PK
name	VARCHAR	100	NOT NULL, UNIQUE
description	TEXT	-	NULL

Таблица 36 - Атрибуты таблицы *approvals*

Атрибут	Тип данных	Размерность	Ограничения
approval_id	BIGSERIAL	-	PK
version_id	BIGINT	-	NOT NULL, FK → concept_versions(version_id)
approver_id	BIGINT	-	NOT NULL, FK → users(user_id)
approval_status_code	VARCHAR	20	NOT NULL, FK → approval_statuses(approval_status_code)
comment	TEXT	-	NULL
sent_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
decided_at	TIMESTAMP	-	NULL

Таблица 37 - Атрибуты таблицы *procurement_statuses*

Атрибут	Тип данных	Размерность	Ограничения
procurement_status_code	VARCHAR	20	PK
name	VARCHAR	100	NOT NULL, UNIQUE
description	TEXT	-	NULL

Таблица 38 - Атрибуты таблицы *procurements*

Атрибут	Тип данных	Размерность	Ограничения
procurement_id	BIGSERIAL	-	PK
application_id	BIGINT	-	NOT NULL, FK → applications(application_id)
specification_id	BIGINT	-	NOT NULL, FK → specifications(specification_id)
procurement_status_code	VARCHAR	20	NOT NULL, FK → procurement_statuses(procurement_status_code)
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
required_date	DATE	-	NULL
return_reason	TEXT	-	NULL

Таблица 39 - Атрибуты таблицы *procurement_items*

Атрибут	Тип данных	Размерность	Ограничения
procurement_item_id	BIGSERIAL	-	PK
procurement_id	BIGINT	-	NOT NULL, FK → procurements(procurement_id)
material_id	BIGINT	-	NOT NULL, FK → materials(material_id)
quantity	NUMERIC	(12,3)	NOT NULL, CHECK (quantity > 0)
required_date	DATE	-	NULL

Для таблицы *procurement_items* устанавливается ограничение *UNIQUE* (*procurement_id*, *material_id*).

Таблица 40 - Атрибуты таблицы *audit_logs*

Атрибут	Тип данных	Размерность	Ограничения
log_id	BIGSERIAL	-	PK
application_id	BIGINT	-	FK → applications(application_id)
user_id	BIGINT	-	NOT NULL, FK → users(user_id)
action	VARCHAR	255	NOT NULL
old_value	TEXT	-	NULL
new_value	TEXT	-	NULL
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Таблица 41 - Атрибуты таблицы *notifications*

Атрибут	Тип данных	Размерность	Ограничения
notification_id	BIGSERIAL	-	PK
user_id	BIGINT	-	NOT NULL, FK → users(user_id)
application_id	BIGINT	-	FK → applications(application_id)
message	TEXT	-	NOT NULL
status	VARCHAR	20	NOT NULL, DEFAULT 'NEW'
created_at	TIMESTAMP	-	NOT NULL, DEFAULT CURRENT_TIMESTAMP
read_at	TIMESTAMP	-	NULL

4. Индексы физической модели

Для системы целесообразно создать следующие индексы.

Таблица 42 - Индексы базы данных

Индекс	Таблица	Поля	Назначение
idx_applications_status	applications	status_code	Фильтрация заявок по статусу
idx_applications_customer	applications	customer_id	Поиск заявок покупателя
idx_applications_manager	applications	manager_id	Поиск заявок менеджера
idx_applications_designer	applications	designer_id	Поиск заявок дизайнера
idx_applications_created_at	applications	created_at	Фильтрация по периоду
idx_concept_versions_concept	concept_versions	concept_id	Получение истории версий
idx_approvals_version	approvals	version_id	Получение согласований версии
idx_procurements_application	procurements	application_id	Получение закупок заявки
idx_audit_logs_application	audit_logs	application_id	Получение истории заявки
idx_audit_logs_created_at	audit_logs	created_at	Поиск действий за период
idx_notifications_user	notifications	user_id	Получение уведомлений пользователя

5. Нормализация и хранение данных

Физическая модель преимущественно соответствует третьей нормальной форме (3НФ).

Справочные значения вынесены в отдельные таблицы:

- roles;
- application_statuses;
- approval_statuses;
- procurement_statuses;
- material_categories.

Позиции спецификаций и закупок также вынесены в отдельные таблицы, что исключает хранение повторяющихся групп значений.

Осознанная денормализация на данном этапе не применяется, поскольку заявленный объём до 100 000 заявок не требует усложнения структуры для повышения производительности.

Файлы концепций физически хранятся вне PostgreSQL, а база данных содержит только сведения о файле и путь к нему.

6. Автогенерация идентификаторов

Для технических идентификаторов используется тип BIGSERIAL, обеспечивающий автоматическую генерацию последовательных значений PostgreSQL.

При этом пользовательский идентификатор заявки хранится отдельно в поле:

application_code VARCHAR(30) UNIQUE NOT NULL

и формируется согласно ранее определённой схеме:

APP-YYYY-NNNNN

7. Диаграмма развертывания

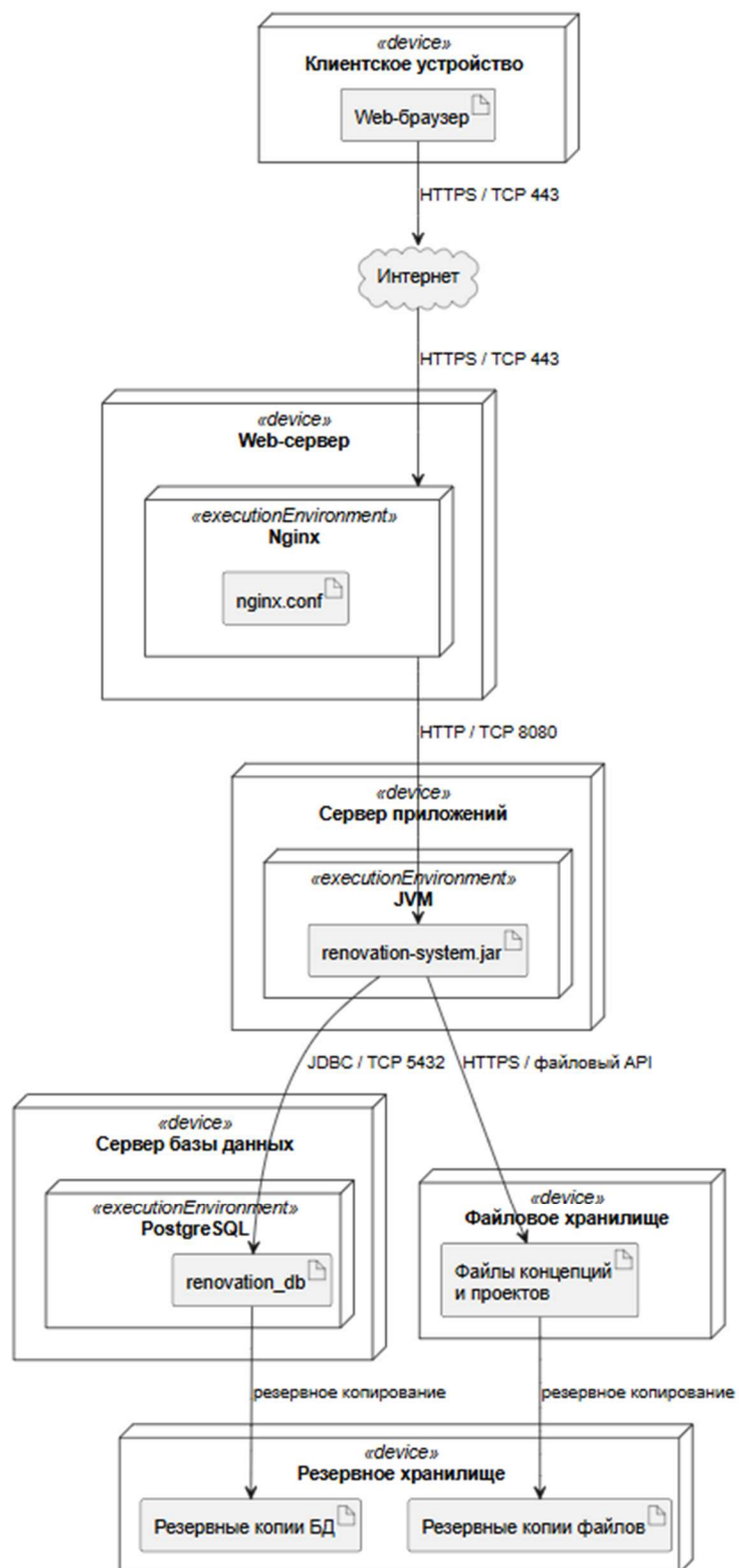


Рисунок 23 - Диаграмма развертывания проектируемой информационной системы

8. Согласованность физической и логической моделей

Проверка физической модели показала, что основные сущности логической модели получили соответствующие таблицы физической БД.

Таблица 43 - Соответствие логической и физической моделей

Логическая сущность	Физическая таблица
Пользователь	users
Роль	roles
Объект ремонта	repair_objects
Заявка	applications
Статус заявки	application_statuses
Концепция	concepts
Версия концепции	concept_versions
Файл проекта	project_files
Спецификация	specifications
Позиция спецификации	specification_items
Материал	materials
Категория материала	material_categories
Согласование	approvals
Статус согласования	approval_statuses
Закупочная потребность	procurements
Статус закупочной потребности	procurement_statuses
Позиция закупки	procurement_items
Журнал действий	audit_logs
Уведомление	notifications

Связи логической модели также сохраняются:

- 1:N реализуются внешним ключом на стороне «многие»;
- 1:1 между заявкой и концепцией обеспечивается UNIQUE для `concepts.application_id`;
- 1:1 между версией концепции и спецификацией обеспечивается UNIQUE для `specifications.version_id`;
- связи материалов со спецификациями и закупками реализуются отдельными таблицами позиций.

Вывод

В ходе выполнения практического занятия разработана физическая модель базы данных информационной системы сопровождения заявки на ремонт квартиры.

Для реализации базы данных выбрана PostgreSQL. Для каждой сущности логической модели определены физические таблицы, типы данных, первичные и внешние ключи, ограничения NOT NULL, UNIQUE, CHECK и DEFAULT. Определены дополнительные индексы, обеспечивающие эффективный поиск заявок, согласований, уведомлений и записей аудита.

Разработана диаграмма развертывания, отражающая размещение клиентского web-интерфейса, Nginx, Java-приложения Spring Boot, PostgreSQL, файлового и резервного хранилищ.

В результате физическая модель согласована с ранее разработанной логической моделью и выбранной в практическом занятии №6 архитектурой системы.

Практическое занятие № 8

Цель занятия:

Смоделировать взаимодействие компонентов проектируемой информационной системы и внешних сервисов, а также определить меры информационной безопасности для этих взаимодействий без разработки собственной спецификации API.

Постановка задачи:

1. Определить основные интеграционные сценарии информационной системы.
2. Описать участников и границы взаимодействия внутренних компонентов и внешних сервисов.
3. Разработать UML Sequence Diagram для двух ключевых интеграционных сценариев.
4. Определить роли пользователей, права и ограничения доступа.
5. Сформировать матрицу разграничения доступа RBAC.
6. Определить основные меры информационной безопасности.

Результат работы:

1. Определение интеграционных сценариев

Проектируемая информационная система реализована как web-приложение с серверной частью на Java и Spring Boot. Для хранения структурированных данных используется PostgreSQL, а файлы концепций размещаются в отдельном файловом хранилище. В процессе обработки заявки также предусмотрено взаимодействие отдела снабжения с поставщиками.

В предыдущих работах для системы были определены функции хранения проектных файлов и взаимодействия с поставщиками при формировании закупочной потребности.

Для моделирования выбраны два интеграционных сценария:

Сценарий 1 - загрузка файла концепции в выделенное файловое хранилище.

Участники:

- дизайнер;
- web-интерфейс;
- Spring Boot-приложение;
- PostgreSQL;
- файловое хранилище.

Назначение сценария - обеспечить сохранение файлов концепции в выделенном файловом хранилище отдельно от базы данных. В PostgreSQL сохраняются только метаданные файла и ссылка на объект.

Сценарий 2 - передача закупочной потребности внешнему поставщику.

Участники:

- сотрудник снабжения;
- web-интерфейс;
- Spring Boot-приложение;
- PostgreSQL;
- внешняя информационная система поставщика.

Назначение сценария - передать сформированную закупочную потребность поставщику и получить результат обработки.

Для обоих сценариев взаимодействие осуществляется по защищённому соединению HTTPS. Для структурированных сообщений используется JSON. Методические материалы требуют указывать протоколы и форматы именно на таком уровне, без разработки собственной API-спецификации.

2. Интеграционный сценарий загрузки файла концепции

Дизайнер формирует новую версию концепции и прикрепляет проектный файл. Web-интерфейс передаёт файл серверному приложению. Spring Boot проверяет права пользователя, тип и размер файла, после чего передаёт файл в файловое хранилище.

При успешном сохранении файловое хранилище возвращает идентификатор или путь к объекту. После этого серверное приложение сохраняет сведения о файле в таблице `project_files`.

Если файловое хранилище недоступно, сведения о файле в базе данных не создаются, а пользователь получает сообщение об ошибке.

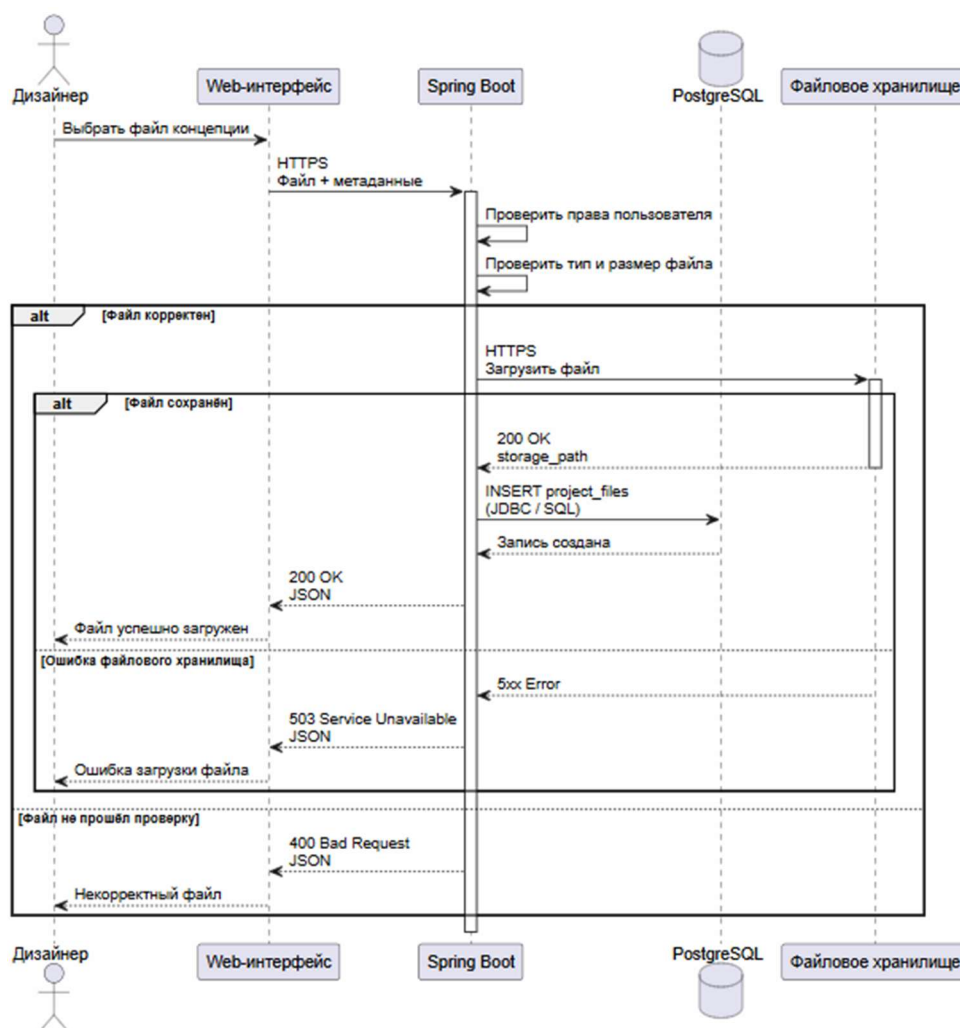


Рисунок 24 - Диаграмма последовательности загрузки файла концепции

3. Интеграционный сценарий передачи закупочной потребности

После завершения всех обязательных согласований система формирует закупочную потребность на основании утверждённой спецификации.

Сотрудник снабжения проверяет перечень материалов и инициирует передачу данных поставщику. Spring Boot получает позиции закупочной потребности из PostgreSQL и формирует JSON-сообщение.

Внешняя система поставщика обрабатывает запрос и возвращает результат.

Возможны три основных результата:

- закупочная потребность принята;
- закупочная потребность отклонена;
- внешний сервис недоступен.

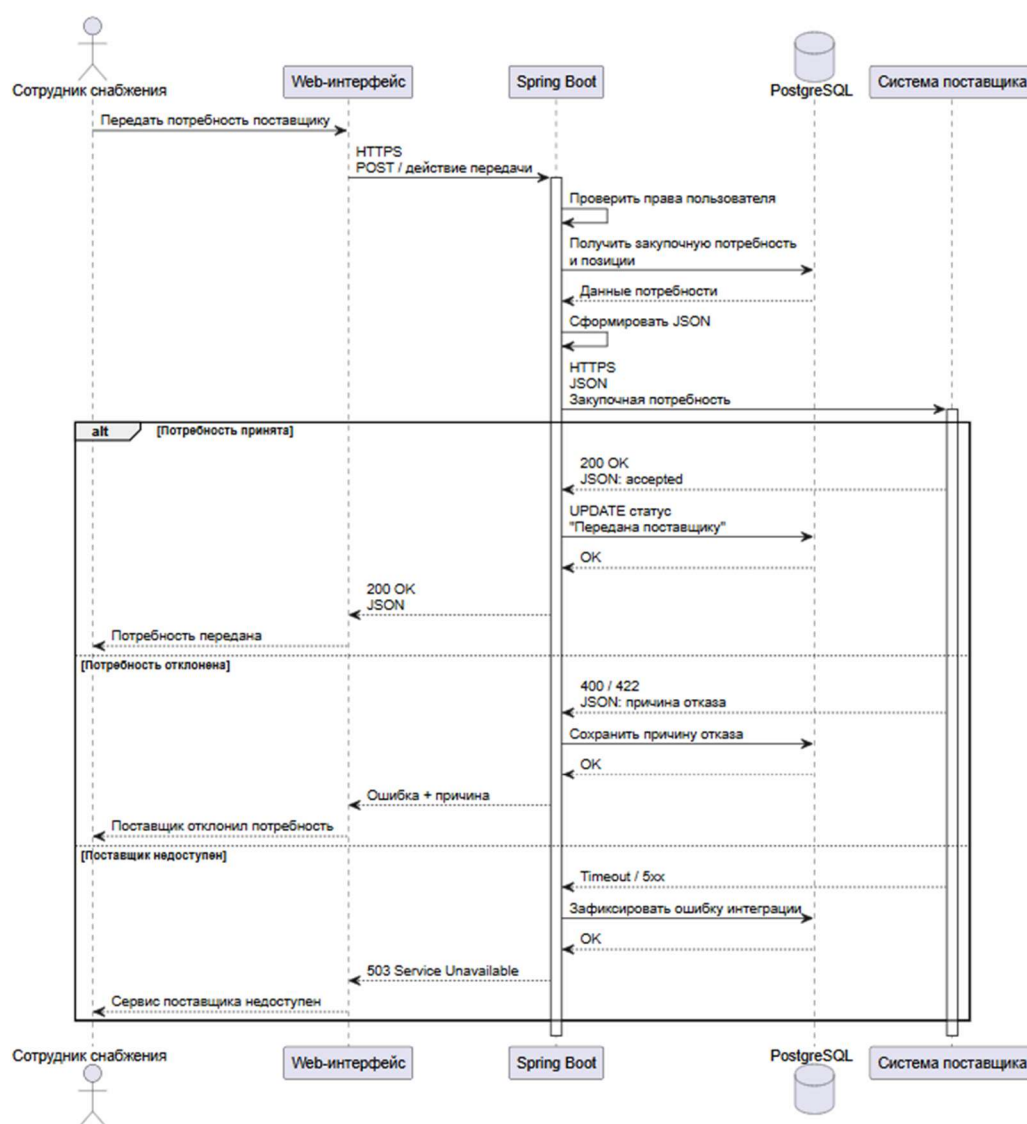


Рисунок 25 - Диаграмма последовательности передачи закупочной потребности

4. Разграничение прав пользователей

В проектируемой системе используется ролевая модель управления доступом - RBAC.

В рамках проектируемой информационной системы каждому пользователю назначается одна основная роль, определяющая доступный набор операций. Роль пользователя хранится в поле `users.role_code`

В ранее разработанном кодификаторе были определены следующие роли:

- CUS - покупатель;
- MAN - менеджер;
- DES - дизайнер;
- PRO - проектный отдел;
- CON - отдел строительства;
- SUP - снабжение;
- ANA - аналитик;
- ADM - администратор.

РВАС предполагает, что разрешения назначаются роли, а не конкретному пользователю. Методические указания требуют придерживаться принципа минимальных привилегий: по умолчанию операция запрещена и разрешается только необходимый набор действий.

Таблица 44 - Разграничение прав пользователей

Роль	Доступные операции	Основные ограничения
Покупатель	Создание заявки, просмотр заявки, согласование концепции	Только собственные заявки
Менеджер	Просмотр и изменение заявок, назначение дизайнера, изменение статусов	Нет управления системными ролями
Дизайнер	Работа с концепциями, версиями, файлами и спецификациями	Только назначенные ему заявки
Проектный отдел	Просмотр концепции, принятие решения по согласованию	Нет редактирования концепции
Отдел строительства	Просмотр концепции, принятие решения по согласованию	Нет редактирования концепции
Снабжение	Просмотр утверждённых заявок, работа с закупочной потребностью	Только заявки, переданные в снабжение
Аналитик	Просмотр аналитических данных и формирование отчётов	Нет изменения заявок
Администратор	Пользователи, роли, справочники, аудит	Не изменяет бизнес-решения согласующих

5. Матрица разграничения доступа RBAC

В соответствии с методическими материалами используются операции:

- C - Create;
- R - Read;
- U - Update;
- D - Delete;
- X - Execute.

Таблица 45 - Матрица RBAC

Роль	Заявки	Концепции и версии	Согласование	Спецификации	Закупки	Аналитика	Пользователи и роли	Справочники	Аудит
Покупатель	C/R ¹	R	R/X	R	-	-	-	R	-
Менеджер	R/U	R	R	R	R	R	-	R	R
Дизайнер	R	C/R/U ²	R	C/R/U ²	-	-	-	R	R ²
Проектный отдел	R	R	R/X	R	-	-	-	R	-
Отдел строительства	R	R	R/X	R	-	-	-	R	-
Снабжение	R	R	R	R	C/R/U/X ³	R	-	R	R ³
Аналитик	R	R	R	R	R	R/X	-	R	-
Администратор	R	R	R	R	R	R	C/R/U/D	C/R/U/D	R/X

Ограничения:

¹ Покупатель получает доступ только к собственным заявкам.

² Дизайнер может изменять концепции и спецификации только по заявкам, на которые он назначен.

³ Сотрудник снабжения работает только с закупочными потребностями, переданными в его функциональную область.

6. Реализация разграничения доступа

В Java-приложении контроль доступа предлагается реализовать с использованием **Spring Security**.

Проверка выполняется на нескольких уровнях:

Уровень web-интерфейса. Пользователю отображаются только доступные ему разделы и действия.

Уровень REST-контроллеров. Перед выполнением операции проверяется наличие необходимой роли.

Например:

```
@PreAuthorize("hasRole('DES')")
```

для операций дизайнера.

Для операций, доступных нескольким ролям:

```
@PreAuthorize("hasAnyRole('MAN', 'ADM')")
```

Уровень бизнес-логики. Дополнительно проверяется принадлежность объекта пользователю.

Например, одного наличия роли CUS недостаточно для просмотра любой заявки: должно выполняться условие:

```
application.customer_id = current_user.user_id
```

Это обеспечивает строковое ограничение доступа.

7. Меры информационной безопасности

Для проектируемой информационной системы предусмотрены следующие меры защиты.

Таблица 46 - Меры информационной безопасности

Направление	Мера
Аутентификация	Вход пользователей по индивидуальной учётной записи
Хранение паролей	Пароли хранятся только в виде стойкого хеша с солью
Авторизация	RBAC на основе ролей Spring Security
Ограничение данных	Покупатель получает доступ только к собственным заявкам
Защита канала	Все внешние соединения выполняются по HTTPS/TLS
Доступ к БД	PostgreSQL недоступен напрямую из сети Интернет
Доступ к файлам	Файлы выдаются только после проверки прав пользователя
Проверка файлов	Контроль размера и допустимого типа загружаемых файлов
Валидация данных	Серверная проверка всех входящих данных
Защита API	Проверка аутентификации и прав для каждого защищённого запроса
Аудит	Фиксация пользователя, времени, действия и изменённых значений
Защита секретов	Пароли БД и ключи внешних сервисов не хранятся в исходном коде
Резервирование	Регулярное резервное копирование PostgreSQL и файлов
Защита от перебора	Ограничение числа повторных попыток входа
Минимальные привилегии	Пользователю выдаются только права, необходимые его роли

Эти меры продолжают ранее сформулированные нефункциональные требования проекта: индивидуальная аутентификация, проверка прав при каждой операции, хранение аудита и резервное копирование данных.

8. Защита интеграционных взаимодействий

Для взаимодействия Spring Boot с внешними сервисами применяются следующие правила:

Передача данных осуществляется только по HTTPS. Незашифрованное HTTP-соединение для внешних интеграций не используется.

Внешний сервис должен проходить аутентификацию. Для межсистемного взаимодействия применяются отдельные технические учётные данные или токены доступа.

Секреты не хранятся в исходном коде приложения. Они передаются через переменные окружения или защищённое конфигурационное хранилище.

Входящие и исходящие данные проверяются. Ответ внешнего сервиса не считается доверенным автоматически.

Ошибки внешнего сервиса обрабатываются отдельно. Недоступность поставщика или файлового хранилища не должна приводить к повреждению внутренних данных.

Интеграционные операции регистрируются в журнале. Для критичных действий сохраняются время запроса, результат и пользователь, инициировавший операцию.

9. Схема защиты доступа

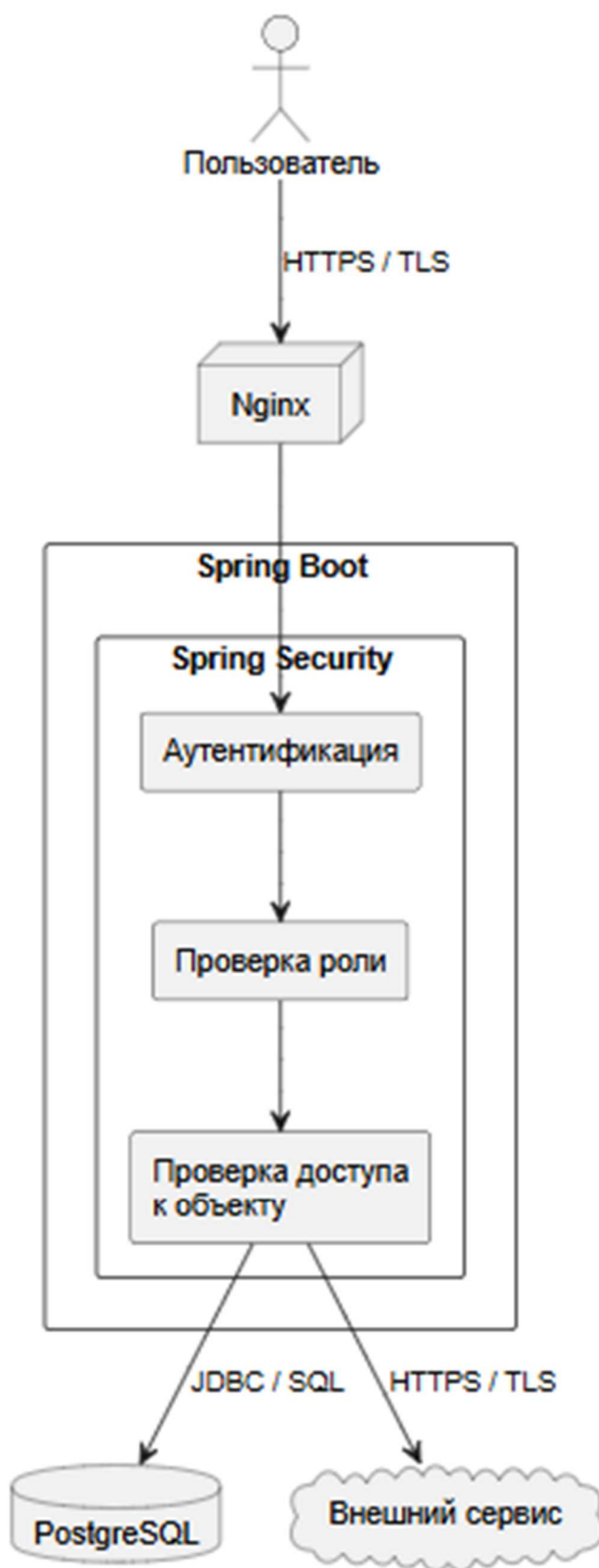


Рисунок 26 - Схема защиты доступа в информационной системе

Вывод

В ходе выполнения практического занятия были определены основные интеграционные взаимодействия проектируемой информационной системы.

Для сценариев загрузки проектных файлов и передачи закупочной потребности внешнему поставщику разработаны диаграммы последовательности, отображающие участников, порядок обмена сообщениями, используемые протоколы и обработку возможных ошибок.

Для управления доступом сформирована ролевая модель RBAC, включающая роли покупателя, менеджера, дизайнера, проектного отдела, отдела строительства, снабжения, аналитика и администратора. Для каждой роли определены доступные ресурсы, операции и ограничения.

Определены меры информационной безопасности, включающие аутентификацию, авторизацию на основе Spring Security, защищённый обмен по HTTPS, серверную валидацию данных, аудит действий пользователей, безопасное хранение секретов и резервное копирование.

Разработанные решения обеспечивают контролируемое взаимодействие внутренних компонентов системы с внешними сервисами и ограничивают доступ пользователей в соответствии с их функциональными обязанностями.

Список использованных источников и литературы:

1. Кириллина Ю. В., Трушин С. М. Разработка обеспечивающих подсистем. Практикум. М.: РТУ МИРЭА, 2025. <https://ibc.mirea.ru/books/SHARE/6385/>.
2. Кириллина Ю. В., Лентяева Т. В. Реинжиниринг бизнес-процессов: учебно-методическое пособие. М.: РТУ МИРЭА, 2025. <https://ibc.mirea.ru/books/SHARE/6432/>.
3. Object Management Group. Business Process Model and Notation (BPMN), Version 2.0.2. <https://www.omg.org/spec/BPMN/2.0.2/> (дата обращения: 15.09.2026).
4. Object Management Group. Unified Modeling Language (UML), Version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/> (дата обращения: 15.09.2026).
5. Planoplan. Спецификация: справочный материал. <https://planoplan.com/ru/library/tools/toolkit/specification/> (дата обращения: 04.09.2026).
6. Remplanner. Программа для планирования ремонта и дизайна интерьера квартиры. <https://remplanner.ru/programma-dlya-dizajna-interera-kvartiry/> (дата обращения: 04.09.2026).
7. Битрикс24. Бизнес-процессы: интерфейс и возможности. <https://helpdesk.bitrix24.ru/open/21290220/> (дата обращения: 04.09.2026).
8. PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/> (дата обращения: 15.09.2026).
9. Spring. Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/> (дата обращения: 15.09.2026).
10. Spring. Spring Framework Reference Documentation. <https://docs.spring.io/spring-framework/reference/> (дата обращения: 15.09.2026).

11. Spring. Spring Data JPA Reference Documentation.
<https://docs.spring.io/spring-data/jpa/reference/> (дата обращения: 15.09.2026).
12. Spring. Spring Security Reference Documentation.
<https://docs.spring.io/spring-security/reference/> (дата обращения: 15.09.2026).
13. Hibernate. Hibernate ORM Documentation.
<https://hibernate.org/orm/documentation/> (дата обращения: 15.09.2026).
14. Apache Software Foundation. Apache Maven Documentation.
<https://maven.apache.org/guides/> (дата обращения: 15.09.2026).
15. NGINX. NGINX Documentation. <https://nginx.org/en/docs/> (дата обращения: 15.09.2026).
16. JetBrains. IntelliJ IDEA Documentation.
<https://www.jetbrains.com/help/idea/> (дата обращения: 15.09.2026).
17. PlantUML. PlantUML Language Reference Guide.
<https://plantuml.com/guide> (дата обращения: 15.09.2026).
18. OWASP Foundation. Authentication Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 15.09.2026).
19. OWASP Foundation. Authorization Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (дата обращения: 15.09.2026).
20. OWASP Foundation. Transport Layer Security Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html (дата обращения: 15.09.2026).